ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

Corso di Laurea in Ingegneria e Scienze Informatiche

Progettazione di un configuratore di vendite

Tesi di laurea in:
Basi di dati

Relatore
Prof.sa Annalisa Franco

Candidato
Cavina Mattia

III Sessione di Laurea
Anno Accademico 2025-2026

Sommario

To be written.

Indice

Elenco delle figure

List of Listings

6.1 Esempio di query LINQ 31
7.1 Esempio di trigger 41
8.1 Esempio di query Queryable 47

Elenco delle tabelle

Capitolo 1

Introduzione

Il progetto di tesi è volto alla progettazione ex-novo del programma di vendita dell'azienda Cepi S.p.a., azienda con una produzione di tipo **ETO!** (**ETO!**) specializzata nella progettazione di impianti di stoccaggio custom. La tipologia di produzione **ETO!** ha portato ad un profondo studio della situazione odierna, delle sue esigenge attuali e future tenendo aperte le possibilità di ampliamenti. L'analisi ha comportato un sostanziale lavoro di programmazione e raccolta dati per coprire nel modo più ampio i bisogni aziendali. Questa prima fase è stata condotta inizialmente attraverso la selezione di figure chiave scelte per mansione o/e ruolo, che potessero dare una visione dei bisogni del proprio o altrui reparto e successivamente sottoponendoli ad interviste. Sono state condotte 14 interviste, di cui 8 in modalità vis a vis, mentre le restanti 6 attraverso la dimostrazione dell'attuale applicativo utilizzato. Il risultato delle interviste condotte, ha fatto emergere attuali e nuove necessità architetturali da affrontare come il cross-platform, l'utilizzo offline del database e la necessità di costruire offerte in modo più agile e strutturato. Questa prima fase ha inoltre sottolineato come il progetto non debba solo sostituire l'attuale sistema, ma debba inserirsi in un contesto di rinnovamento più ampio, coordinandosi con altri programmi aziendali e relativi database in ottica "**SRP!** (**SRP!**)" eleggendo dei master data che siano i soli responsabili della creazione di certi dati. Il **SRP!** ha plasmato notevolmente la costruzione del database ricreando, dove possibile, le strutture pre-esistenti da cui si necessitava recuperare i dati mediante reverse engineering. **??** L'intero sistema è stato pensato per ap-

poggiarsi all'infrastruttura esistente aziendale, come SQLServer[**?**], ma studiando delle logiche di flessibilità e ampliamento dinamico studiando accuratamente le relazioni tra le tabelle. Si è reso necessario cercare e scegliere inoltre un framework per la parte grafica che potesse soddisfare il deployment cross-platform. Questo requisito è emerso in modo chiaro dalle interviste, in quanto ci sono situazioni in cui il personale aziendale si trova a dover lavorare in mobilità, come le fiere di settore. La costruzione delle interfacce grafiche, è stata supportata dal sito "Figma"[**?**] che ha permesso una progettazione in ottica mobile first.**??** La scelta accurata dei framework e le strutture dati nella fase iniziale, ha pesato molto in termini di tempo e progettazione, per garantire al richiedente un sistema che non rischiasse né il debito tecnico, né la preclusione a sviluppi futuri. La pianificazione dei primi test con gli utenti, dei rilasci delle demo e del metodo di progettazione sono state cruciali per dettare tempi e priorità degli item dovendo coprire in modo esaustivo almeno il core del progetto.

Capitolo 2

Situazione As-Is

2.1 Premessa

La situazione infrastrutturale aziendale è stata studiata dal sottoscritto nel corso del tirocinio formativo. La raccolta dei dati è stata svolta mediante interviste e studio dell'attuale sistema informatico. Nelle sezioni che seguono introdurrò brevemente l'As-Is.

2.2 Situazione infrastrutturale aziendale

L'infrastruttura informatica aziendale si appoggia su server fisici allocati internamente e gestiti mediante Active Directory. I server fisici sono, a loro volta virtualizzati per gestire separatamente le memorie in base alla finalità garantendo isolamento delle informazioni e permessi di accesso. Ogni reparto ha accesso a determinati server virtuali in cui gestisce in autonomia i propri contenuti. I database dei principali programmi sono gestiti all'interno di un server dedicato dove è installato un motore SQLServer.[**?**]. I principali database aziendali sono:

- **AdHoc[?]:** l'**ERP!** (**ERP!**) aziendale, programma che gestisce le attività aziendali come gli approvvigionamenti e la movimentazione dei prodotti, le vendite, le fatturazioni nonché i cicli produttivi.

- **Fusion[?]:** il **PLM!** (**PLM!**) aziendale, programma volto alla gestione dei prodotti meccanici, dalla loro creazione all'archiviazione dei disegni con relative distinte dei componenti. Altre funzioni applicate sui prodotti sono la gestione delle revisioni o versionamenti, ed infine la loro fine produzione.
- **BombConfine:** database ponte tra i database esterni e quello dell'**ERP!**. All'interno sono presenti tabelle di frontiera scritte da programmi esterni e che il gestionale usa per importare i dati.

La sicurezza aziendale è sostenuta da **NAT!** (**NAT!**) e Firewall che impediscono l'accesso dall'esterno se non tramite **VPN!** (**VPN!**). L'infrastruttura aziendale è visibile in figura **??**.

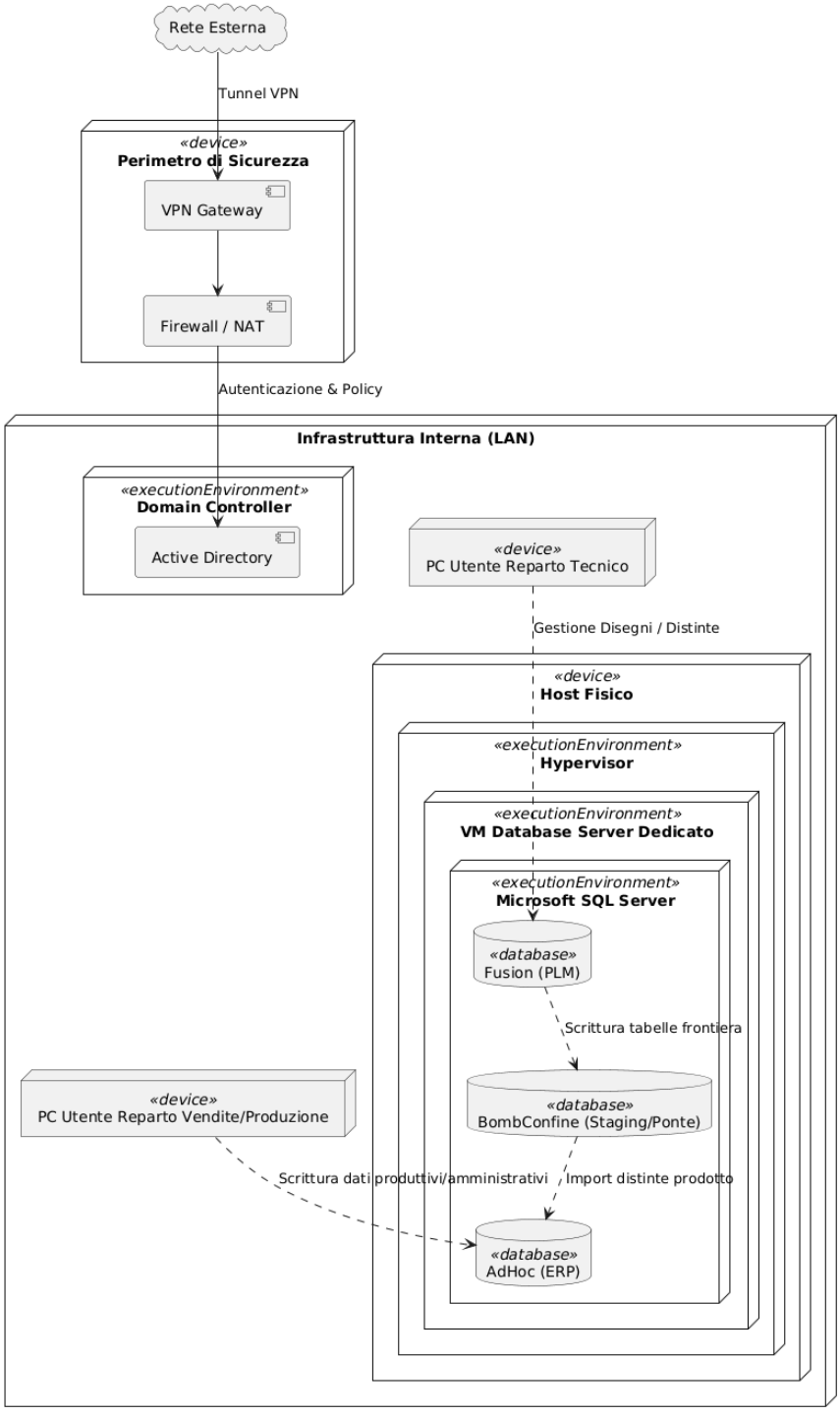


Figura 2.1: Schema infrastruttura aziendale

2.3 Attuali software utilizzati

Nel capitolo precedente abbiamo fatto cenno ai database ed ai relativi programmi che gestiscono quanto concerne l'**ERP!** e il **PLM!**. Per quanto riguarda le vendite, punto focale dell'intervento richiesto, l'azienda utilizza attualmente un'applicazione sviluppata internamente. L'attuale sistema sviluppato in Access, programma della suite Microsoft Office, comprende al suo interno sia il frontend che il backend. Il software è stato implementato circa 30 anni fa ed ha subito numerosi aggiornamenti e modifiche nel tempo per adattarsi alle esigenze commerciali e all'evoluzione del mercato. L'applicazione consente di creare, modificare e gestire i contratti con i clienti, includendo funzionalità quali la gestione degli sconti, la personalizzazione delle offerte e la stampa dei documenti. Tuttavia, nonostante le funzionalità implementate, il software presenta significative criticità che ne limitano l'efficienza e l'usabilità, come confermato dalle interviste condotte con gli utenti. Tali criticità includono instabilità operativa, mancanza di standardizzazione, codifiche obsolete, difficoltà negli aggiornamenti di prezzo e assenza di integrazione con altri sistemi aziendali. Di conseguenza, è indispensabile sviluppare un nuovo programma che affronti tali problematiche e migliori l'esperienza degli utenti, preservando nel contempo le funzionalità essenziali del sistema attuale. Il programma Access è stato fondamentale in questi anni per l'utilizzo in zone offline. Questa funzionalità è stata favorita dalla natura del programma stesso, il quale racchiude in un solo file database e frontend, copiandolo in locale e sincronizzandolo una volta tornato in sede.

2.4 Organizzazione aziendale e flussi di lavoro

Il flusso lavorativo è risultato non essere un aspetto secondario nello sviluppo richiesto. Esso si deve interporre tra il reparto commerciale, ovvero addetti alle vendite, i reparti progettuali e produttivi come uffici tecnici, logistica ed amministrazione. L'attuale flusso di lavoro analizzato pone il reparto commerciale come punto di partenza dell'intero flusso che arriva fino al post-vendita. Il contratto, creato da tale reparto, dopo essere stato stipulato viene inviato al reparto amministrativo che ne importa i dati all'interno del gestionale. Simultaneamente esso

viene mandato al reparto project manager che lo revisiona tecnicamente in caso di errori progettuali. Dopo la revisione tecnica, il contratto viene spedito al reparto logistico che preventivamente verifica i dati di destinazione ed organizza le spedizioni sulla base del venduto. Il flusso di lavoro può essere visto in figura **??**. A causa del sistema obsoleto utilizzato ed il debito tecnico accumulato negli anni, l'attuale sistema è isolato rispetto ai programmi utilizzati dall'azienda. Quest'ultimo punto fa parte delle richieste imprescindibili da risolvere. L'integrazione nell'ecosistema informatico passa prima di tutto dall'unificazione dei codici, ovvero quelli a listino con i prodotti a gestionale, ma anche creando un flusso di notifiche tra i vari ambiti.

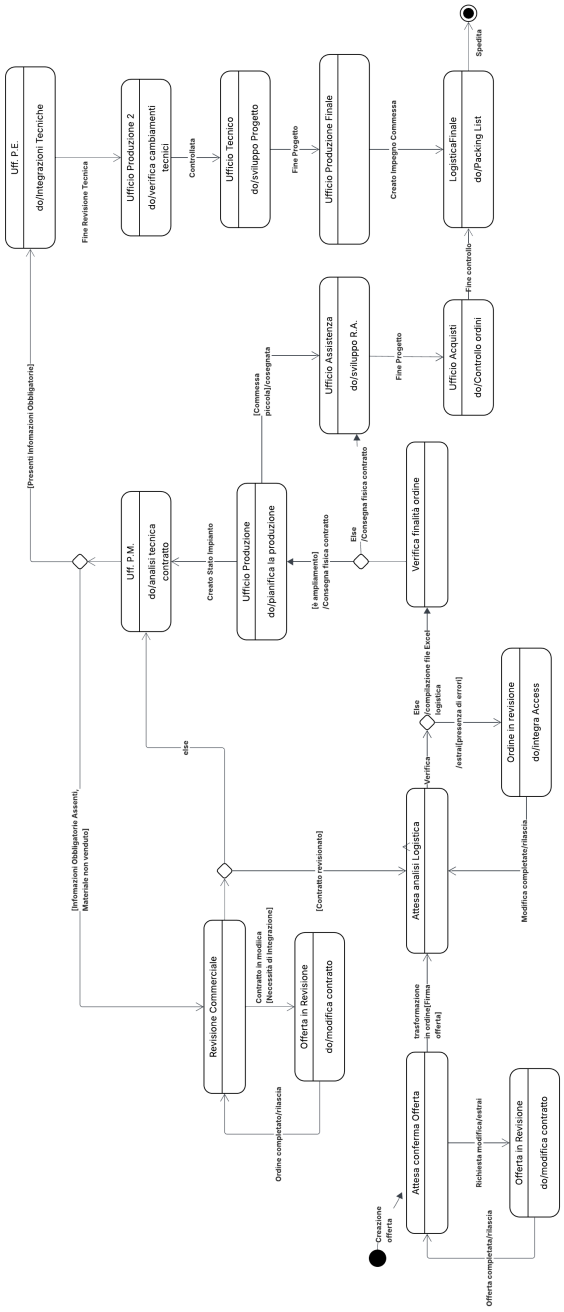


Figura 2.2: Schema degli stati di una offerta commerciale

Capitolo 3

Raccolta dei requisiti

3.1 Scelta dei candidati

La raccolta requisiti si è svolta mediante la selezione di figure aziendali con distinti ruoli ed esperienza. Queste figure selezionate sono state poi sottoposte ad una intervista sia in modalità vis a vis, che in alcuni casi da remoto per favorire la registrazione del meeting. I candidati possono essere raggruppati in soci aziendali, utilizzatori, e non utilizzatori. All'interno degli ultimi due gruppi sono state selezionate, per far fronte ai bias dovuti all'abitudine, candidati sia con esperienza pluriennale che neoassunti. In totale sono state intervistate 14 persone così suddivise:

Utilizzatori abituali:

- **Agenti commerciali esterni (2):** 1 con esperienza limitata, 1 con esperienza consolidata.
- **Commerciali interni (4):** 1 nuovi impianti, 1 ricambi, 2 con competenze trasversali.

Utilizzatori saltuari:

- **Soci con responsabilità (3):** 1 vendite, 1 tecnico, 1 amministrativo.
- **Direttore di produzione (1)**

Non utilizzatori:

- **Impiegati tecnici (4):** 1 project manager, 1 logistica, 1 progettista meccanico, 1 progettista elettrico.

L'ordine delle interviste svolte non è stato lineare, ma si è optato di partire dalla proprietà, per capire la direzione ed aspettative del progetto. Le restanti interviste hanno seguito un ordine logico, ma al contempo sparso, alternando utilizzatori e non utilizzatori, con l'obbiettivo di non saturare ogni singolo reparto. Questa modalità, ha consentito di fare da guida per le interviste successive.

3.2 Modalità di svolgimento interviste

Le interviste, come esplicato in precedenza, si sono svolte singolarmente sia in presenza che da remoto. Con il permesso degli interlocutori sono state registrate, anche solo l'audio, per poi essere rielaborate e trascritte con supporto di **IA!** (**IA!**) generativa. La scelta di utilizzare la modalità fisica o da remoto è stata dettata dalla necessità. Per gli utilizzatori abituali, si è reso necessario documentare visivamente le modalità di utilizzo dell'attuale applicativo. Tramite questa modalità si è potuto raccogliere i flussi di utilizzo e le principali funzioni presenti da mantenere. Nei restanti incontri si è preferito un approccio in presenza con l'interlocutore, ha facilitato la raccolta di requisiti e consigli sul progetto.

Le prime interviste sono state supportate da una scaletta di domande, preparate sulla base dell' AS-IS **??** che hanno guidato l'incontro. Queste domande sono riportate in Appendice **??**. Con il progredire delle interviste, è stato adottato un approccio più flessibile che ha affinato i risultati delle interviste in previsione delle successive. Questa modalità ha consentito di ricavare i requisiti fondamentali ordinati per importanza.

3.3 Risultati interviste condotte

Le interviste, come già accennato, sono state trascritte e rielaborate mediante **IA!** generativa e, successivamente, revisionate manualmente per evitare errori di inter-

pretazione. I risultati delle interviste possono essere visti in modo approfondito nelle seguenti appendici:

- **Appendice ??:** Risultati interviste con la proprietà.
- **Appendice ??:** Risultati interviste con gli utilizzatori.
- **Appendice ??:** Risultati interviste con i non utilizzatori.

Nelle prossime sezioni, verranno riportati i risultati più significativi emersi durante le interviste, raggruppati per macro aree.

3.3.1 Modifiche rispetto all'attuale sistema

Funzionalità da mantenere

Dalle interviste condotte, è emersa l'importanza di mantenere una serie di funzionalità, quali:

- Suddivisione del contratto in capitoli che raggruppano porzioni di esso.
- Gestione delle scontistiche a cascata, ovvero la possibilità di inserirle rispetto alla singola voce contrattuale, nel capitolo, o dell'offerta stessa.
- Gestione di voci o capitoli opzionali, che non concorrono al prezzo offerto, ma ad uno opzionale.
- I campi del database che attualmente concorrono alla creazione dei report, sono da mantenere.
- Creazione reportistica formato PDF del contratto digitale. Da gestire, come ora, il multilingua del report stesso.
- Esportazione all'**ERP!** delle voci contrattuali presenti nell'ordine.

Criticità emerse

Mediante le dimostrazioni di utilizzo si sono potute raccogliere le criticità da attenzionare, come:

- Blocchi frequenti in condizione di concorrenza. Se più utenti utilizzano in contemporanea il file si generano deadlock.
- Non è presente una gestione centralizzata dei listini prodotto. Attualmente si gestisce esternamente tramite file Excel.
- Mancanza di linee guida nella compilazione delle offerte. Si è avanzata l'ipotesi di un configuratore di prodotto che aiuti l'utente.
- Mancanza di gestione dei valori duplicati e controlli sulle foreign key porta a creare clienti e prodotti duplicati.

3.3.2 Richieste avanzate

Gestione ruoli e permessi

La scelta di coprire quanti più reparti possibili, ha fatto emergere la mancanza di gestione dei ruoli. Si è richiesto quindi di attribuire ad ogni utente un ruolo, a sua volta ognuno di essi avrà viste sul database e permessi dedicati. I ruoli individuati sono:

- Admin con visione sull'intero sistema, con possibilità esclusiva di modifica ad alcune tabelle fondamentali.
- Commerciali interni che possono gestire totalmente i clienti e contratti.
- Commerciali esterni all'azienda, esempio agenti a partita IVA, possono vedere solo i propri clienti e contratti.
- Amministrazione contabile, può modificare i dati contabili dei clienti. Questo ruolo può altresì esportare le voci contrattuali all'**ERP!** per gestire la fiscalità.
- Project Manager con compito di revisionare tecnicamente il contratto. Questo compito è volto alla gestione a monte dell'invenduto.
- Visualizzatore senza permessi di scrittura.

Gestione dei contratti

Nella creazione dei contratti, core del progetto, si dovrà sviluppare il vero salto di qualità. Nelle interviste si è cercato di raccogliere le richieste che, anche dai primi rilasci, potessero dare al richiedente un sistema più ottimale. Tali richieste sono:

- Gestione storico dei dati in caso di modifica.
- Gestione di più versioni di uno stesso contratto legati dal medesimo codice progressivo. In caso di approvazione serve far decadere le versioni non approvate.
- Importazione dal **PLM!** di dati tecnici dei prodotti. Questi dati sono fondamentali nei calcoli utili alla contrattazione (Es: Potenza elettrica totale consumata).
- Gestione degli stati di avanzamento di un progetto e relative notifiche.
- Revisione tecnica del venduto in modo da dare la possibilità di proporre all'occorrenza integrazioni contrattuali.

Capitolo 4

Analisi dei requisiti

4.1 Requisiti funzionali

Questa sezione è dedicata alla suddivisione dei dati raccolti in requisiti funzionali. In linea di massima, i requisiti emersi mirano a mantenere la flessibilità del sistema, ma al contempo a garantire la sicurezza dei dati e la loro integrità. Sono emersi infatti, in molteplici punti, la necessità di evitare duplicazioni di dati ed il mantenere un filo logico della vita di una offerta. La concretezza dei dati è da mantenere mediante logiche di controllo ed incrocio dei dati, segnalando o bloccando l'utente in caso di incongruenze. Tra i requisiti emersi, è stata riconosciuta la necessità di gestire i backup dei dati. Il loro scopo è sia per garantire la sicurezza dei dati stessi, sia per permettere un eventuale ripristino in caso di errori. La tabella **??** raccoglie il dettaglio dei requisiti funzionali raccolti, suddividendoli per area del progetto. Insieme al requisito e alla sua descrizione è presente anche l'elenco dei ruoli a cui esso è rivolto.

Area / Modulo	Requisito Funzionale	Descrizione / Specifiche	Ruoli Coinvolti
Sicurezza & Accessi	Profilazione RBAC granulare	Gestione permessi per tabella (lettura/scrittura) tramite tabelle utenti e utenti_tipi.	Tutti i ruoli
Anagrafica Clienti	Controllo anti-duplicazione	Validazione obbligatoria dell'univocità tramite Partita IVA in fase di inserimento.	Amministrazione, Agenti, Commerciali
Anagrafica Clienti	Sedi multiple e tipizzazione	Gestione sedi legali/operative, destinazioni merce separate e distinzione cliente finale / rivenditore.	Amministrazione, Agenti
Anagrafica Clienti	Storico contratti cliente	Consultazione integrata di tutte le offerte e ordini emessi per lo specifico cliente.	Commerciali, Amministrazione
Catalogo Prodotti	Codifica univoca	Allineamento del codice prodotto tra Configuratore, Ad-Hoc ERP e Fusion PLM.	Admin, Commerciali
Catalogo Prodotti	Gestione obsoleti e visibilità	Blocco dei componenti obsoleti rilevati da PLM; flag di visibilità per migrazione progressiva da ERP.	Admin, PM / Engineering
Catalogo Prodotti	Selezione per Macrogruppi	Navigazione guidata ad albero (Macchina → Componenti → Accessori → Ricambi) con vincoli di compatibilità.	Commerciali Nuovi/Ricambi
Catalogo Prodotti	Scheda caratteristiche tecniche	Attributi tabellari strutturati: meccanici, elettrici, pneumatici, logistici e accessori correlati.	Commerciali, PM / Engineering
Catalogo Prodotti	Descrizioni standardizzate	Modello ibrido: voce principale da menu standardizzato (da Fusion) affiancata da campo note libero.	Commerciali, PM / Engineering
Offerte & Pricing	Struttura a Capitoli	Suddivisione delle offerte in capitoli gestiti come sottodistinte personalizzabili.	Commerciali
Offerte & Pricing	Pricing centralizzato e override	Prezzi sincronizzati da Ad-Hoc con possibilità di inserimento manuale per voci speciali o su misura.	Commerciali, Admin
Offerte & Pricing	Scontistica multi-livello	Calcolo sconti combinabili: per riga prodotto, per capitolo, globale (esclusa MDO), globale (inclusa MDO).	Commerciali
Offerte & Pricing	Calcoli tecnici ed economici	Calcolo automatico di potenza elettrica, consumo aria compressa, pesi, volumi e stima costi montaggio/trasporto.	Commerciali
Offerte & Pricing	Vincoli Incoterms e Paesi	Selezione assistita Incoterms (con vincoli UE/extra-UE); adattamento automatico a valuta, lingua e normative (es. UL-CSA).	Commerciali, Logistica
Offerte & Pricing	Alert informativi	Notifiche a video non bloccanti per segnalare scelte non conformi alle specifiche standard d'impianto.	Commerciali
Offerte & Pricing	Versioning e clonazione	Storico revisioni tracciato (Rev.01, Rev.02...) e funzione di duplicazione rapida di offerte esistenti.	Commerciali
Ordini & Workflow	Conversione e congelamento	Blocco del contratto commerciale alla conferma cliente e assegnazione codifica causale ordine.	Commerciali, Amministrazione
Ordini & Workflow	Notifiche multi-reparto	Alert push automatici inviati a Amministrazione, PM, Logistica e Produzione all'apertura dell'ordine.	Sistema (Cross-funzionale)
Ordini & Workflow	Esportazione verso ERP	Invio automatizzato dell'ordine confermato ad Ad-Hoc per avviare la fatturazione.	Amministrazione
Revisione Tecnica	Contratto tecnico parallelo	Gestione di una variante tecnica interna (senza valore legale) gestita dal PM con costificazione separata.	Project Management
Revisione Tecnica	Workflow decisionale modifiche	Bivio post-revisione: approvazione commerciale con nuovo contratto/RDA (impatto alto) o gestione interna (impatto basso).	PM, Commerciale
Stampe & Documenti	Layout di stampa per reparto	Generazione PDF personalizzati per Cliente, Amministrazione e Ufficio Tecnico.	Tutti i reparti
Localizzazione	Supporto multilingua	Interfaccia localizzata e traduzione automatica/manuale delle descrizioni contrattuali.	Commerciali, Agenti

Tabella 4.1: Requisiti funzionali raccolti.

4.2 Requisiti non funzionali

Questa sezione, a differenza della precedente raccoglie i requisiti non funzionali. In linea di massima, i requisiti non funzionali mirano a migliorare l'attuale esperienza utente. Sono da attenzionare in particolare la fluidità del sistema, l'intuitività delle interfacce, e la velocità di esecuzione. Le interfacce devono essere predisposte per device mobili e proporre all'utente solo ciò che il suo ruolo necessita. Tra i requisiti non funzionali, è emerso come sia necessario gestire la raccolta dei dati da altri database, anche a runtime, per garantire la correttezza dei dati e la loro integrità. Un esempio sono le giacenze dei prodotti in magazzino. La tabella **??** seguente, insieme al requisito stesso, raccoglie anche la sua categoria e la sua priorità all'interno dell'implementazione. I requisiti sono ordinati per priorità.

Categoria	Requisito Non Funzionale	Priorità	Specifica Tecnica / Obiettivo
Affidabilità	Eliminazione crash e blocchi	Critica	Stabilità operativa durante inserimento massivo righe, calcoli e stampa, azzerando i blocchi tipici di Access.
Interoperabilità	Connessione nativa ERP	Critica	Integrazione bidirezionale diretta via query SQL Server con Ad-Hoc (anagrafiche, listini ed export ordini).
Disponibilità	Architettura offline (LocalDB)	Alta	Continuità operativa garantita anche in assenza di rete o VPN tramite istanza locale SQL Server LocalDB.
Integrità Dati	Sincronizzazione automatica	Alta	Motore di sync bidirezionale tra LocalDB e SQL Server aziendale con logica di risoluzione conflitti.
Resilienza	Backup automatici in VPN	Alta	Salvataggio automatico del database locale durante le connessioni VPN per prevenire perdite o corruzioni dati.
Integrità Dati	Unificazione codici prodotto	Alta	Coerenza assoluta delle codifiche articolo tra Configuratore, Ad-Hoc ERP e Fusion PLM senza disallineamenti.
Usabilità	Workflow guidati (Wizard)	Alta	Percorsi passo-passo che guidano la compilazione per azzerare omissioni o errori di configurazione commerciale.
Usabilità	Viste orientate al ruolo (RBAC)	Alta	Maschere e sezioni della UI mostrate dinamicamente in base alle prerogative del ruolo utente loggato.
Portabilità	Framework moderno	Media	Architettura per ambiente desktop con predisposizione a target mobile.
Prestazioni	Calcolo istantaneo in-memory	Media	Elaborazione immediata a runtime di formule per potenze, consumi d'aria, pesi, volumi e stime logistiche.
Qualità Processo	Area di staging ordini	Media	Area di transito temporanea per la revisione amministrativa delle conferme d'ordine prima del caricamento contabile.
Evolutività	Predisposizione CAD / PLM	Bassa	Architettura aperta al recupero di disegni tecnici e visualizzazione di modelli 3D direttamente da Fusion.

Tabella 4.2: Requisiti non funzionali raccolti.

4.3 Flussi di lavoro

I flussi di lavoro citati in figura **??** sono risultati complessi e poco efficienti. Si è quindi deciso di inserire una revisione tecnica, da parte di un utente specializzato, degli impianti venduti. Il nuovo diagramma, in figura **??**, mostra un flusso

più lineare che porta a valle meno errori. Determinare i nuovi flussi lavorativi è fondamentale per progettare gli automatismi a cui sono soggette le notifiche intra-reparto.

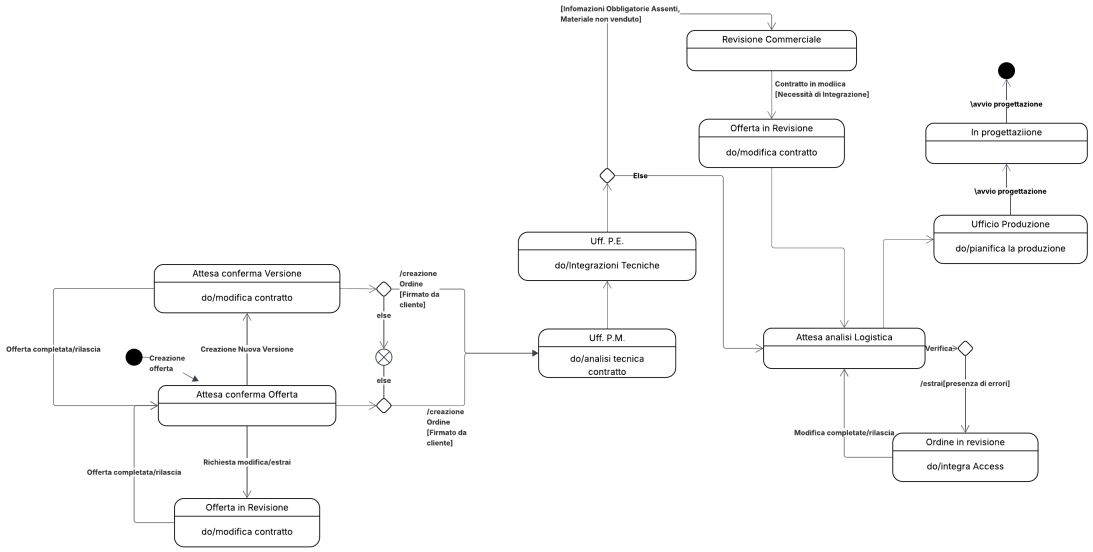


Figura 4.1: Nuovo diagramma degli stati di un contratto

Capitolo 5

Progettazione dell'architettura

5.1 Premessa sulle scelte architetturali

Le scelte architetturali sono dipese in primo luogo dall'infrastruttura aziendale, e successivamente dalle richieste degli utenti durante gli incontri. L'utilizzo di SQLServer, come motore di database, è stato adottato come base di tutte le scelte successive. Tale decisione è stata presa in quanto esso è già presente in azienda, pertanto non richiede ulteriori investimenti. Un'ulteriore motivazione è che tale motore contiene i dati dell' **ERP!** aziendale e del **PLM!** di prodotto. Mantenendo lo stesso motore di database, si è potuto garantire la compatibilità dei dati e la loro integrità. Da questo caposaldo, la ricerca si è indirizzata verso un framework che interagisse sia con SQLServer sia con SQLite.[**?**]. La necessità di operare offline ha guidato la ricerca di un sistema installabile in locale, escludendo così le soluzioni web based. La scelta è quindi ricaduta sulla piattaforma .NET[**?**] con il linguaggio C# che, in quanto parte dell'ecosistema Microsoft, è perfettamente integrata con i database prima citati. La piattaforma .NET è espandibile mediante l'utilizzo di NuGet[**?**], che permette di aggiungere package e quindi funzionalità al progetto. I pacchetti sono sviluppati sia da Microsoft sia da terze parti, e vengono aggiornati e supportati continuamente. Questa manutenzione continua evita di incorrere in debiti tecnici futuri. L'ambiente .NET è nativamente multipiattaforma e questa proprietà soddisfa le necessità dei richiedenti. Secondo le linee guida Microsoft, per lo sviluppo delle view multipiattaforma, è consigliato l'utilizzo del package

NuGet .NET MAUI[**?**]. In fase di compilazione, MAUI converte il codice sorgente in codice nativo per la piattaforma di destinazione. MAUI, integrato con il package NuGet "Community Toolkit Maui"[**?**], consente di costruire le interfacce grafiche in codice C#. Nativamente MAUI supporta solo il linguaggio Markup XAML[**?**].

5.2 Deployment

La scelta degli ambienti di sviluppo precedentemente citati porta a dover considerare il deployment dell'applicativo. Il software verrà installato in locale su ogni dispositivo, sia esso desktop o mobile, e si connetterà al database centrale. All'interno dell'azienda è presente un dominio **AD!** (**AD!**), dove ogni oggetto del dominio ha un **IP!** (**IP!**) statico. Questa configurazione permette la connessione al database, risiedente in un server all'interno del dominio, in modo agile tramite **IP!**, se si è connessi all'interno della **LAN!** (**LAN!**). Lo scambio di dati tra il frontend e il backend avviene mediante protocollo **TCP/IP!** (**TCP/IP!**). Si è valutato, per chi accede dall'esterno del dominio, di utilizzare la rete **VPN!**, stabilendo una connessione virtuale che permette di essere all'interno della **LAN!**. Il database locale, in SQLite, è strutturato come una copia del database centrale. I dati vengono allineati tra i due database mediante un processo di sincronizzazione. In assenza di connessione, verificabile tramite ping, si accede al database locale ed, una volta ristabilita la connessione, i dati vengono sincronizzati. Il diagramma dei deployment è riassunto in figura **??**.

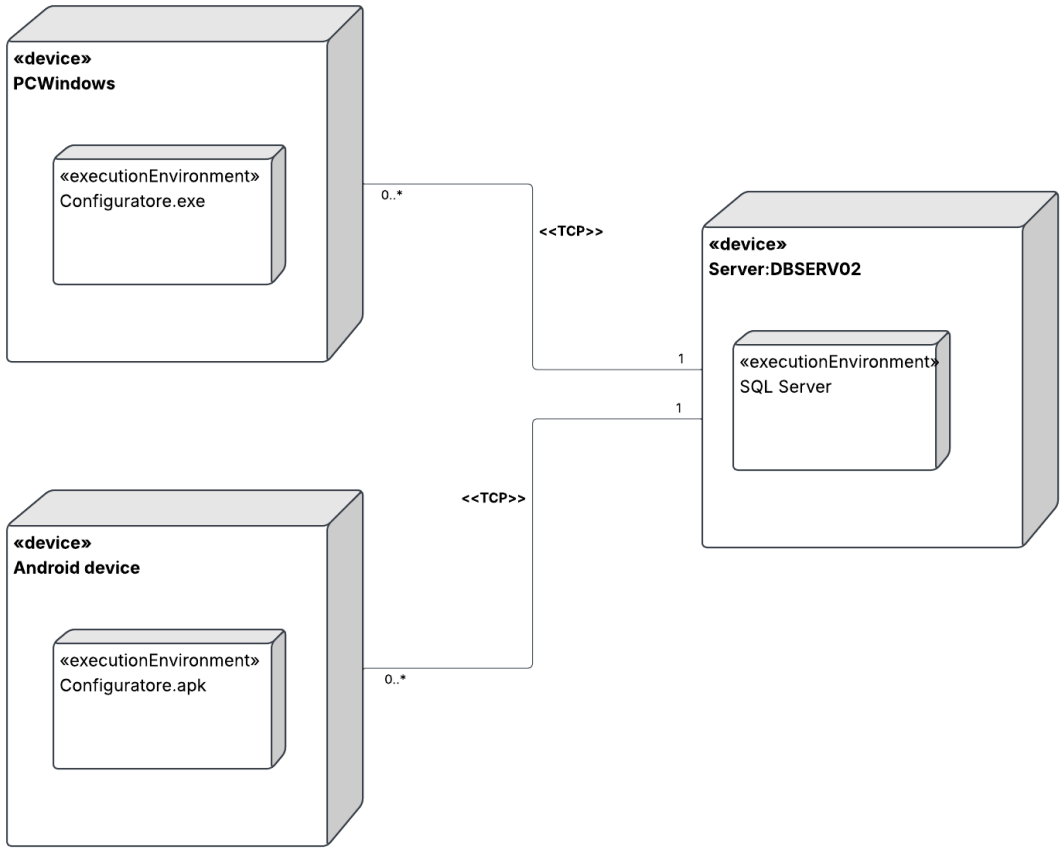


Figura 5.1: Diagramma dei deployment

Nella fase di sviluppo, per testare la struttura del database, è stato scelto di utilizzare una versione di SQLServer locale, denominata SQL Express [**?**]. SQL Express condivide con SQLServer tutte le caratteristiche principali. Questa proprietà ha permesso di costruire query e testare funzionalità localmente in modo compatibile con il database di produzione.

5.3 Package

L'intero software è stato suddiviso in tre distinti package, con responsabilità diverse. Si è cercato di seguire il principio **DRY!** (**DRY!**) e la riusabilità degli stessi.

Il principio **DRY!** consiste nel non ripetere codice già scritto, ma di riutilizzarlo. Questo approccio permette di ridurre la complessità del software, facilitando la manutenzione e l'evoluzione futura. Il mantenimento del principio **DRY!** necessita di continua rifattorizzazione del codice.

5.3.1 Interfaccia al database

Il package di interfaccia al database, denominato "DatabaseManagementPackage", ha il compito di gestire tutti gli aspetti legati a quest'ultimo, come:

- gestione delle connessioni ai vari database, come SQLite e SQLServer o nel caso dei test, SQL Express;
- gestione delle transazioni;
- gestione dei permessi utente, bloccando le operazioni non consentite;
- conversione, mediante **ORM!** (**ORM!**), delle tabelle del database in entity;
- mappatura di entity in file neutri, proteggendo il database;
- gestione di Repository per ogni entità, che espongono le operazioni **CRUD!** (**CRUD!**);
- gestione di una suite di test per verificare le operazioni precedenti. Questa suite è risultata fondamentale nell'evoluzione del database, validando i cambiamenti apportati;
- esporre al di fuori del package solo file neutri e la classe manager del database.

Nell'implementazione si è utilizzato il pacchetto NuGet **EF Core!** (**EF Core!**) [**?**]. Questo modulo permette una gestione efficiente delle operazioni di accesso ai dati, consentendo di astrarre le tabelle del database in classi di entity. Le entity sono state a loro volta mappate in **DTO!** (**DTO!**). Un **DTO!** è un design pattern architetturale ideato per ottimizzare lo scambio di informazioni tra diversi layer di un'applicazione. Opera come un aggregatore leggero che raggruppa i dati necessari per una specifica comunicazione (come il traffico client-server). Privo di

logica di business o comportamenti applicativi, il **DTO!** espone esclusivamente le proprietà richieste dal caso d'uso: questa semplificazione minimizza la dimensione del payload, riduce le chiamate di rete e la latenza complessiva, ottimizzando le prestazioni del sistema. La traduzione da entity a **DTO!** protegge i dati del database, evitando che vengano esposti direttamente. [**?**] Questa seconda mappatura è stata realizzata mediante il package "Mapster" [**?**], appartenente a terze parti. La mappatura entity-**DTO!** è stata realizzata in modo bidirezionale in quanto, in alcuni casi, è necessario convertire un **DTO!** in entity per poterlo salvare nel database.

5.3.2 Libreria di componenti view

La creazione di interfacce grafiche è stata supportata da un package dedicato, che racchiude in sé factory di componenti, costanti e utilità. Il package, denominato "ViewComponentLibrary", è stato concepito a sé stante per favorire un suo impiego in implementazioni future. La libreria si appoggia a sua volta al package NuGet "Community Toolkit Maui"[**?**]. Questo package permette di scrivere le interfacce grafiche in codice C# Markup, evitando quindi l'utilizzo del linguaggio XAML[**?**], standard di sviluppo per le interfacce MAUI.

5.3.3 Modulo main e view

Il package principale del progetto è dedicato alla gestione dei componenti e dei flussi di lavoro. Il package, denominato "ConfiguratoreCommerciale", ha numerosi compiti, tra cui la gestione della Dependency Injection, la creazione delle view e dei rispettivi ViewModel. In questo package risiedono infatti i ViewModel, che si interpongono tra le view e i repository, esponendo i dati tramite proprietà osservabili (Observable Properties). La Dependency Injection, invece, permette di iniettare le dipendenze necessarie alle classi, come ad esempio i repository, le classi di logging o le view stesse se registrate. Le classi di view, utilizzando i componenti della libreria, vengono realizzate tenendo conto del dispositivo di destinazione. La differenziazione è permessa dall'utilizzo della compilazione condizionale, distinguendo il codice da compilare in base alla piattaforma di destinazione. Un esempio di tale condizione è il seguente:

```
#if ANDROID
    // codice per piattaforma Android
#elseif WINDOWS
    // codice per piattaforma Windows
#endif
```

Questa possibilità permette anche di eseguire parti di codice solo in fase di DEBUG. Gli aspetti generali gestiti dal package possono essere riassunti come segue:

- creazione delle interfacce grafiche;
- traduzione a compile time del codice C# in XAML;
- traduzione a compile time in elementi nativi per la piattaforma di destinazione;
- gestione delle interfacce mediante ViewModel, che a loro volta tramite proprietà osservabili modificano la view in risposta ad eventi;
- gestione della navigazione tra le interfacce grafiche;
- gestione dei permessi, mostrando solo i comandi che competono al ruolo dell'utente;
- gestione della Dependency Injection mediante la classe *MauiProgram.cs*;
- creazione, in fase di avvio, di una cache in **RAM!** (**RAM!**) di dati immutabili durante la sessione, come tabelle di lookup.

Il diagramma dei package è riassunto in figura **??**.

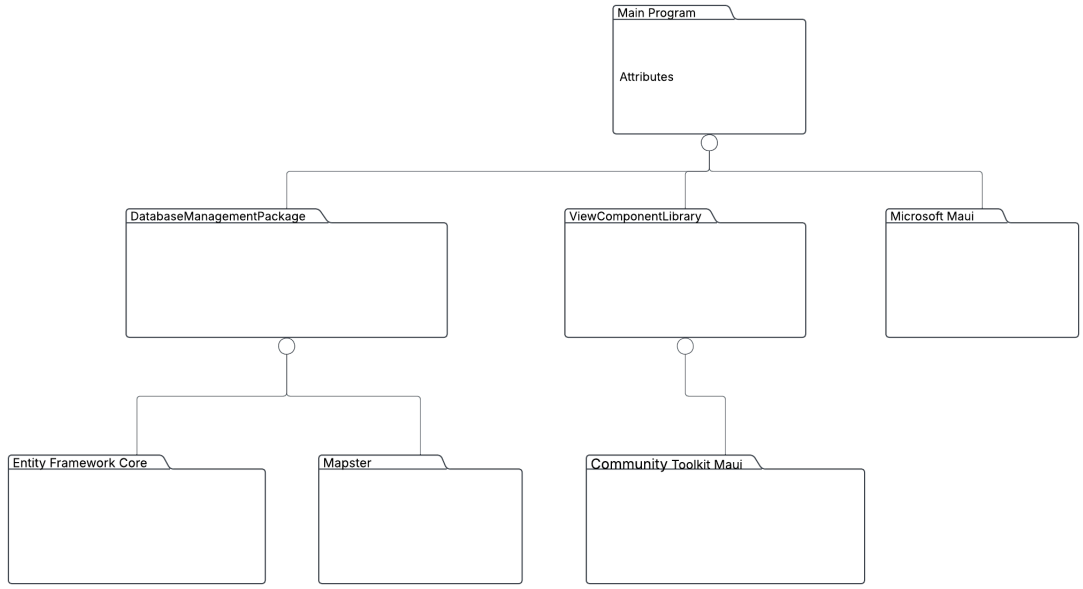


Figura 5.2: Diagramma dei package

Capitolo 6

Scelte progettuali

6.1 Scelta modello di sviluppo

Il modello di sviluppo scelto è basato sulla necessità di prototipare e testare con gli utenti le funzioni nell'arco dell'intero sviluppo. Si è quindi deciso di adottare un modello agile, supportato dalla creazione di test delle funzioni e dei prototipi delle interfacce. La progettazione è stata iterativa e incrementale, seguendo i canoni dell'agilità. Tutti i metodi di interazione col database sono stati testati da appositi Unit Test che ne validassero il comportamento. I test di regressione hanno permesso di convalidare le modifiche apportate al database e di conseguenza al software. Le interfacce grafiche, al contrario, non sono state supportate da test, ma prototipate e approvate da utenti scelti. La rifattorizzazione del codice è stata affrontata con regolarità seguendo il principio **DRY!**. Nel seguire tale principio, si sono create più classi di Factory e classi di utilità che unificassero quanto più possibile il codice comune.

6.2 Suddivisione e contenuto dei rilasci

Le tempistiche dei rilasci sono state decise insieme al Project Manager aziendale. Il collega è stato di supporto nella stima dei tempi di sviluppo e nella conversione del database esistente, guidando lo sviluppo attraverso le tecniche professionali. Nel primo rilascio, si è scelto di includere tutti i requisiti già presenti nel precedente

applicativo. In tale rilascio, con consegna stimata per gennaio 2027, sono presenti le seguenti funzionalità:

- gestione utenti e permessi;
- anagrafica clienti;
- creazione delle offerte;
- versioning offerte;
- creazione ordini;
- import ed export dati, da e verso altri database;
- creazione reportistica;
- gestione multilingua dell'interfaccia applicativa;
- prototipo mobile;
- modulo di configurazione prodotto.

Si è scelto di non includere la funzionalità offline, in quanto essa necessita di un database centrale stabile. La funzionalità offline verrà implementata nel secondo rilascio. Questa fase avverrà una volta che il software sarà in produzione, per poter contare su un database pressoché stabile. Nel secondo rilascio saranno quindi presenti:

- gestione revisione tecnica e differenze con contratto firmato;
- inserimento di calcoli automatici sulla base delle caratteristiche del venduto (es. potenza elettrica totale assorbita);
- gestione multi-valuta per ordini internazionali;
- apertura dei disegni PDF dal server del **PLM!**;
- report avanzati;
- implementazione database locale con logiche di sincronizzazione.

6.3 Scelta framework ORM

All'interno della sezione sull'architettura, è stata citata la scelta del framework **EF Core! ??**. Questo framework ha permesso di mappare le tabelle del database in classi mediante la tecnica **ORM!**. La tecnica **ORM!** permette di gestire il database con un approccio **OOP!** (**OOP!**), astraendo le tabelle in classi. Le mappature relazionali a oggetti **ORM!** collegano le applicazioni scritte in un linguaggio di programmazione **OOP!** ad un database sottostante. Questi tipi di applicazioni e database vengono spesso utilizzati insieme, ma ognuno gestisce i dati in modo diverso. L'aggiunta di una mappatura relazionale a oggetti consente a ciascuno di inviare dati in un formato comprensibile all'altro. È possibile modificarne uno senza influire sull'altro. [**?**] I legami tra le entità sono gestiti mediante la classe `Context`. L'**ORM!** ha permesso, con l'utilizzo di script PowerShell, di ricreare in modo rapido tutte le entity e il context a fronte di cambiamenti del database. Tale approccio può essere definito database-first, in quanto la progettazione avviene principalmente sul database e il software ne eredita le modifiche. Le queries sono state scritte tramite le funzioni **LINQ!** (**LINQ!**), che hanno permesso di formularle in linguaggio C#. Un esempio di query è riportato nella funzione **??**.

Listing 6.1: Esempio di query LINQ

```
1    public class Example{
2  public async Task<ClientiDTO?> GetCustomerDetailsById(int customerId,
3      CancellationToken cancellationToken = default)
4  {
5      // Check if the user has permission to read ClientiDTO
6              await _permissionTable.TryGetTablePermission(typeof(ClientiDTO),
                   TypeOfOperation.READ,
7          _utils.GetCurrentUserRoleId()).ConfigureAwait(false);
8
9      Clienti? customerDetails = await _db.Clientis
10         .Where(c => c.CliId == customerId)
11         .Include(c => c.ClientiDestinazionis.Where(cd => cd.Deleted == false))
12         .ThenInclude(d => d.CdeNazioneNavigation)
13         .Include(c => c.ClientiDestinazionis.Where(cd => cd.Deleted == false))
14         .ThenInclude(d => d.CdeTipoNavigation)
15         .Include(c => c.CliBancaSedeNavigation)
16         .Include(c => c.CliRiferimentoEsternoNavigation)
17         .Include(c => c.CliRiferimentoInternoNavigation)
18         .Include(c => c.CliTipoNavigation)
19         .Include(c => c.DocumentiDocUtilizzatoreNavigations.Where(d => d.
               Deleted == false))
```

```
20              .Include(c => c.CliCorriereNavigation)
21              .Include(c => c.CliImballoNavigation)
22              .AsNoTracking()
23              .FirstOrDefaultAsync(cancellationToken)
24              .ConfigureAwait(false);
25
26  // Map the retrieved customer details to a ClientiDTO object and return it
27          return _mapper.Map<ClientiDTO?>(customerDetails!);
28      }
29          }
```

In ottica di offline, le queries in **LINQ!** sono compatibili con il database locale SQLite se si limitano le funzioni utilizzate a quelle supportate da esso. Le funzioni compatibili sono ad esempio: *Where, Select, OrderBy, GroupBy, Join, Distinct.*

Uno degli aspetti negativi di **LINQ!** è la lentezza con grandi volumi di dati. Lo svantaggio è dovuto alla traduzione del codice in query compatibili con **SQL!** (**SQL!**). Si è valutato però, in caso si presentino problemi di performance, di ottimizzare il database mediante indici in prima battuta e, successivamente, ottimizzando le singole query. **EF Core!** permette di ottimizzare le queries scrivendole in **SQL!** puro, bypassando la traduzione da **LINQ!**.

6.4 Scelta del framework grafico

Il framework grafico scelto aveva come obiettivo soddisfare il requisito della portabilità e dell'offline. Le scelte vagliate sono state:

- frontend web based, con l'utilizzo di **HTML!** (**HTML!**), **CSS!** (**CSS!**) e **JS!** (**JS!**) e interrogazione del database centrale mediante **API!** (**API!**);
- utilizzo del framework MAUI, nativamente cross-platform e integrato perfettamente con il backend.

L'opzione web based, con interfacce facilmente adattabili a qualsiasi dispositivo, ha presentato due svantaggi principali. La difficoltà principale risiedeva nell'utilizzo in assenza di connessione. In questo scenario, per funzionare, richiedeva che i file e il database fossero presenti sul dispositivo. Questa caratteristica rende difficoltoso gestire gli aggiornamenti. Il secondo motivo, non trascurabile per le tempistiche,

è l'utilizzo di più linguaggi di programmazione. Quest'ultimo non massimizza l'esperienza acquisita con l'implementazione degli altri moduli. Il framework MAUI di .NET sopperisce a entrambi i problemi. Il programma è fisicamente installato sui dispositivi e il loro aggiornamento può essere gestito centralmente all'interno di uno store Microsoft. Essendo basato su .NET e avvalendosi del framework "Community Toolkit Maui", le interfacce sono scritte in C#.

6.5 Scelta del pattern di progettazione

Insieme a MAUI, Microsoft suggerisce l'utilizzo del pattern **MVVM!** (**MVVM!**).[**?**] Il modello **MVVM!** consente di separare in modo pulito le logiche di business e le relative interfacce utente. La gestione separata risolve numerosi problemi, quali la possibilità di ridisegnare le interfacce utente senza dover modificare le classi che le governano. Un secondo vantaggio è la possibilità di consentire agli sviluppatori e ai progettisti dell'interfaccia utente di collaborare più facilmente durante lo sviluppo delle rispettive parti di un'app.[**?**] Il modello **MVVM!** è composto da tre elementi fondamentali che interagiscono come in figura **??**:

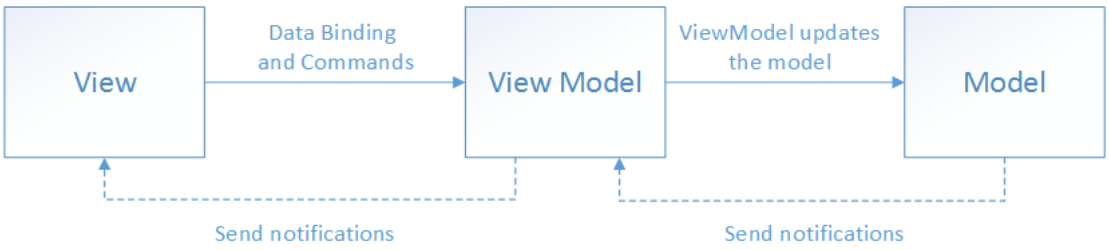


Figura 6.1: Diagramma funzionamento MVVM

Questo modello, separando la logica della view da quella di business, ha permesso di prototipare le interfacce nei primi rilasci e in futuro riprogettarle migliorando l'accessibilità, ovvero la capacità di un prodotto di essere utilizzato da persone con disabilità. Migliorarla vuol dire aggiungere valore e rimuovere le barriere che potrebbero impedire l'interazione con esso. Un altro aspetto migliorabile è l'**UX!** (**UX!**), ovvero l'esperienza dell'utente nell'interazione con il prodotto.

Capitolo 7

Progettazione del database

7.1 Esportazione vecchio database

La creazione della nuova base di dati è partita dall'esportazione e dallo studio dell'attuale database Access. L'esportazione è avvenuta per mezzo di **SSMA!** (**SSMA!**), che ha permesso di esportare solo la struttura esistente da Access in SQLServer. **SSMA!** è uno strumento progettato per automatizzare la migrazione del database a SQL Server, da Microsoft Access, Db2, MySQL, Oracle e SAP ASE. [**?**] Ottenuta l'attuale struttura, essa ha subito un processo di ristrutturazione che ha portato ad avere un database più leggibile, manutenibile e performante.

7.2 Denominazione tabelle e campi

Il primo intervento effettuato è stato quello di rinominare le tabelle ed i campi del database. Le modifiche addottate sono state le seguenti:

- Denominare le tabelle in minuscolo, separati da underscore, ad esempio: *nome_tabella*.
- Denominare i campi delle tabelle in minuscolo, separati da underscore,con il prefisso della tabella per renderlo univoco. Un esempio di campo è: *prefTabella_nomeCampo*.

- Le chiavi primarie di ogni tabella sono degli indici progressivi, rendendo unica ogni riga. Il campo segue il pattern *prefTabella_id*.

Insieme alle modifiche sopra citate, sono stati aggiunti dei campi di audit, come data ed utente di creazione, ma anche data e utente di modifica. Queste informazioni permettono di avere un tracciamento delle modifiche e di chi le ha effettuate. Un ulteriore campo comune aggiunto è *deleted*, booleano per la gestione della cancellazione logica. Si è deciso di non cancellare mai fisicamente i dati, ma di marcare le righe come cancellate per facilitare il ripristino dei dati in caso di errore. Questi campi trasversali sono fondamentali per la gestione dei dati e la manutenzione del database.

7.3 Utilizzo di SSIS nel processo di migrazione

I successivi processi di modifica e popolamento del database sono stati gestiti tramite **SSIS!** (**SSIS!**). [**?**] Questo strumento permette la creazione e l'esecuzione di pacchetti di query, che possono essere schedulati o eseguiti manualmente. Mediante interfaccia grafica è possibile creare flussi di query, con l'opportunità di gestire eccezioni, condizioni di esecuzione e logiche di flusso. Il principale utilizzo di **SSIS!** è stato quello di organizzare un flusso di query per la creazione e popolamento del database. Il primo flusso gestito è il seguente:

1. crea se non esiste il database;
2. se presenti le tabelle, le elimina;
3. crea le tabelle seguendo un ordine consono alle foreign key;
4. popola le tabelle di lookup;
5. popola le tabelle con dati del vecchio database;
6. popola le tabelle restanti con dati di test.

Tramite questo flusso è possibile creare un database pronto all'uso, con dati di test e dati reali, ricostruito con facilità in caso di errori o modifiche. Nelle fasi più

avanzate, alle operazioni prima citate, sono state aggiunte quelle per la creazione delle tabelle e dei trigger di backup, per la gestione dello storico dei dati. A database quasi pronto, si è proceduto alla creazione di un secondo flusso di query per l'importazione dei dati da altri database, come ad esempio **ERP!** e **PLM!**.

7.4 Ristrutturazione database

7.4.1 Eliminazione ridondanze e normalizzazione

La fase di studio dell'attuale database ha portato alla luce diverse ridondanze e dati non normalizzati. All'interno erano presenti più tabelle con lo stesso contesto di utilizzo o tabelle non utilizzate. Si è quindi proceduto all'eliminazione di tali tabelle ed alla normalizzazione delle restanti in **2NF!** (**2NF!**). La **2NF!** è una forma di normalizzazione delle tabelle del database, che richiede che ogni attributo non chiave sia completamente dipendente dalla chiave primaria. Un'altra modifica sostanziale è stata l'unificazione di tabelle similari in un'unica tabella, discriminate mediante campo *prefTabella_tipo* proveniente da un lookup dedicato. Un esempio di questo contesto sono le tabelle di "Offerta" ed "Ordine", dove una è la diretta evoluzione della precedente e mantengono gli stessi dati. Le due tabelle sono state unite sotto il cappello della tabella "Documenti", discriminata solo dal campo *doc_tipo*. In queste tabelle, in particolare, era importante tenere traccia del legame Padre-Figlio. È stato aggiunto un campo *doc_documento_padre* che, mediante chiave importata, permettesse di ricostruire il legame attraverso un auto join. Il metodo così pensato, in ottica di ampliamento, permette la gestione di ulteriori tipologie di documenti senza cambiare la struttura. Un esempio è l'aggiunta della tipologia di documento "Tecnico", che sarà l'evoluzione del documento "Ordine" e conterrà la revisione tecnica dell'ordine stesso.

7.4.2 Creazione tabelle di lookup

Come già prima accennato, le tabelle di lookup sono state create per discriminare le tipologie di dati. Tali tabelle sono state pensate per garantire una estedibilità futura e far fronte a modifiche non previste. Queste tabelle sono considerate fisse,

e non possono essere modificate dall'utente finale, ma solo da un amministratore tramite query. Esse, con al più un campo descrittivo, rendono la gestione dei dati più semplice, potendo filtrare le informazioni in base al loro tipo. Un vantaggio, a livello di software, è il poterle mappare in classi di enumerazione, rendendo il codice più leggibile ed evitando non necessari join nelle query. Un ultimo vantaggio, essendo appunto fisse, è il poterle caricare in memoria all'avvio dell'applicativo, evitando di interrogare il database ad ogni richiesta.

7.4.3 Aggiunta tabelle non presenti

Gestione dei prodotti

Il database originale utilizzava i prodotti come oggetti atomici, senza la possibilità di gestire i loro componenti. Si è quindi deciso di creare le distinte prodotto. La tabella *Prodotti* è stata messa in auto join con cardinalità n ad n. Tale relazione a livello di schema logico è stata realizzata mediante la tabella *Prodotti_composizioni*. La tabella in questione aggiunge, oltre alle chiavi dei prodotti, anche il quantitativo della riga stessa e la posizione di apparizione. Lo schema relazionale risultante è visibile in figura **??**.

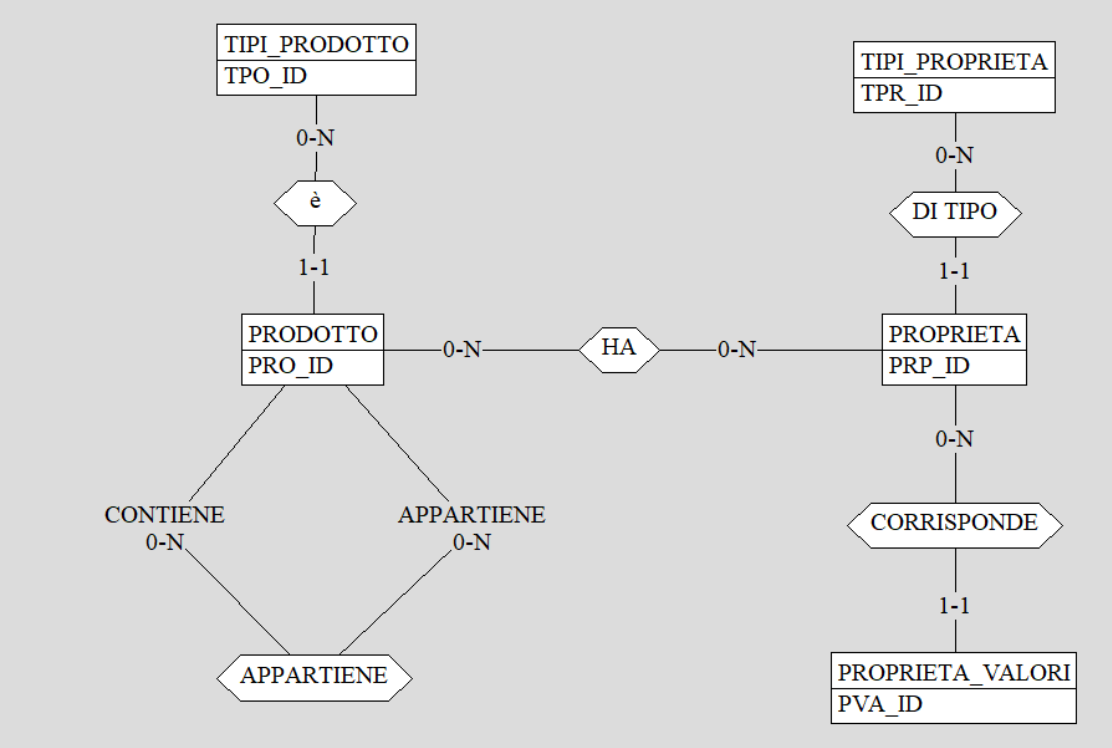


Figura 7.1: Schema **ER!** (**ER!**) dei prodotti

La gestione, così pensata, permette di effettuare ricerche approfondite. Un esempio è quello della estrapolazione dei componenti di ricambio di un determinato prodotto, che possono risiedere anche a più livelli di profondità. Questo sistema, insieme ad appositi lookup, ha inoltre permesso la costruzione del modulo "Configuratore" per la configurazione di prodotto. **??**.

Tabelle gestione lingue

Si è deciso di aggiungere delle apposite tabelle per la gestione delle traduzioni di campi del database. Le tabelle predisposte contengono al loro interno la chiave della tabella di riferimento, il campo da tradurre e la chiave importata dalla tabella "Lingue". Quest'ultima tabella non era presente nella soluzione di partenza. La gestione così pensata permette di estendere le traduzioni a più lingue. La gestione delle reportistiche dei documenti è gestita mediante una tabella dedicata. Questa

tabella mappa le tipologie di reportistica richieste dal cliente con le apposite lingue concordate. Capita infatti di avere ad esempio manuali in lingua inglese, ma etichette sui macchinari con la lingua del luogo. Il diagramma **ER!** delle lingue è visibile in figura **??**.

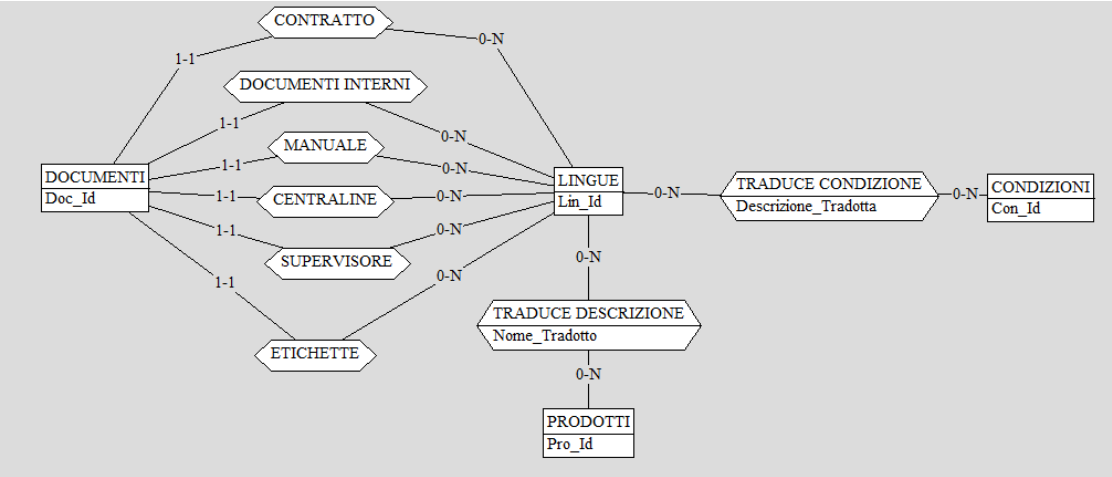


Figura 7.2: Schema **ER!** delle lingue

Creazione tabelle di backup

Si è scelto di gestire solo un ristretto numero di essenziali tabelle, come ad esempio quelle dei documenti, dei prodotti, delle righe sia dei documenti che dei prodotti. La gestione non è stata capillare su tutte le tabelle, in quanto si sono scelte solo le tabelle più centrali del dominio. Questo approccio è volto ad evitare di appesantire il database con tabelle, dati e trigger non necessari. Le strutture delle tabelle di backup rispecchiano quelle delle tabelle originali, ma senza vincoli né chiavi primarie o secondarie, garantendo così la possibilità di salvare più modifiche della stessa tupla. Il popolamento avviene mediante trigger. Essi scattano ad ogni operazione di `UPDATE` eseguita sulla tabella originale. Tramite query ed il campo di audit *data_modifica* si è quindi in grado di ricostruire lo storico dei dati e di ripristinare eventuali errori. Il trigger è marcato come `AFTER UPDATE`. In combinazione con il dato temporaneo `deleted` permette di salvare il dato prima della

modifica, garantendo la possibilità di ripristinare lo stato precedente. Un esempio di trigger è visibile in listato **??**.

Listing 7.1: Esempio di trigger

```
CREATE   TRIGGER trg_prodotti_update
ON [dbo].[prodotti]
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO [dbo].[prodotti_backup] (
        [pro_id], [pro_descrizione], [pro_codice], [pro_prezzo],
        [pro_potenza_installata], [pro_aria_compressa], [pro_peso_a_vuoto],
        [pro_volume], [pro_flag_ricambio], [pro_flag_obsoleto],
        [pro_tipo], [pro_tipo_merceologia], [pro_note],
        [synced_on], [creation_date], [creation_user],
        [last_edit_date], [last_edit_user], [deleted]
    )
    SELECT
        [pro_id], [pro_descrizione], [pro_codice], [pro_prezzo],
        [pro_potenza_installata], [pro_aria_compressa], [pro_peso_a_vuoto],
        [pro_volume], [pro_flag_ricambio], [pro_flag_obsoleto],
        [pro_tipo], [pro_tipo_merceologia], [pro_note],
        [synced_on], [creation_date], [creation_user],
        [last_edit_date], [last_edit_user], [deleted]
    FROM deleted;
END;
```

Tabelle di utilità

Nel database sono state incluse delle tabelle di utilità, che non hanno un legame diretto con le altre tabelle, ma saranno utilizzate per la gestione e manutenzione del database. Queste tabelle sono:

- *Parametri*: questa tabella contiene solo la versione del database corrente. In caso di aggiornamenti del database centrale il software in locale si accorge della differenza e si allinea alla versione più recente. L'aggiornamento avverrà eseguendo le query presenti nella tabella *Query*.
- *Query*: questa tabella raccoglie le query necessarie per aggiornare la struttura del database locale con quella in remoto da una versione all'altra.

Gestione dei permessi

La gestione dei permessi è stata realizzata mediante una tabella dedicata, denominata *Tabelle*. Essa associa al nome di una tabella del database un valore corrispodente al livello massimo di ruolo per eseguire lettura o scrittura su tale tabella. I ruoli sono definiti ed ordinati dal più dominante al meno dominante, all'interno della tabella "Tipi_utente". Si ha quindi 0 per l'Admin e 100 per il visualizzatore, dando modo in futuro di aggiungere altri ruoli intermedi. Questo metodo rende flessibile la gestione dei permessi di accesso alle tabelle, potendole modificare anche da apposita interfaccia grafica. Un esempio della interfaccia è visibile in figura **??**.

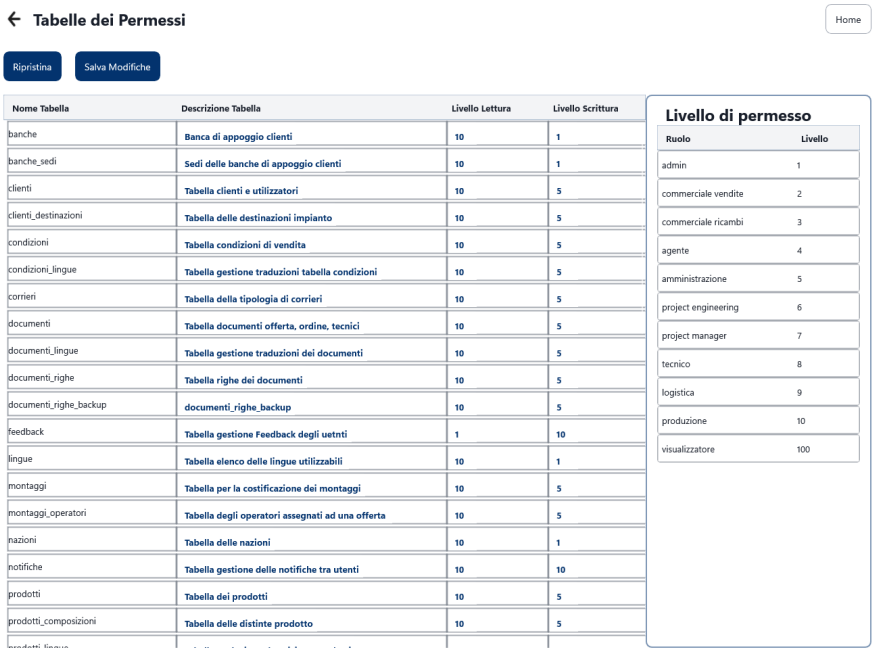


Figura 7.3: Interfaccia di gestione dei permessi

In ottica offline, questa tabella permette di mappare, solo quello che l'utente può modificare in locale, evitando di eseguire query su tabelle non permesse offline.

7.4.4 Importazione dei dati e stored procedure

Come da analisi, è stato richiesto di prelevare e sincronizzare alcuni dati da altri database. Prima della costruzione delle vere e proprie query di sincronizzazione, si sono studiate le tabelle di provenienza per ricostruire un ambiente similare, facilitando la costruzione delle query. In particolare, dal database Fusion, si è ricostruita la struttura di assegnazione dinamica delle proprietà ad un prodotto. Tale struttura, ponendo la cardinalità n ad n tra prodotti e proprietà con aggiunta del campo valore, permette l'assegnazione e la creazione di nuove proprietà senza modificare la struttura del database. Alla tabella "Proprietà" è associata la tabella "Proprietà_valori" per la gestione dei valori standard. Il diagramma **ER!** è mostrato in figura **??**. Una volta ricostruite le strutture si sono create le seguenti stored procedure:

- Da **ERP!** → importazione ed aggiornamento delle anagrafiche dei prodotti.
- Da **ERP!** → importazione ed aggiornamento delle anagrafiche dei listini.
- Da **ERP!** → importazione delle distinte dei prodotti.
- Da **PLM!** → importazione ed aggiornamento delle proprietà dei prodotti.
- Da **PLM!** → importazione dei valori standard associati alle proprietà dei prodotti.

Tutte le stored procedure hanno al loro interno dei controlli che permettono di attuare le modifiche solo se presenti dei cambiamenti, evitando di sovraccaricare il database con operazioni non necessarie. Oltre a ciò saranno da prevedere altre stored procedure di importazione per la gestione di altri dati, come ad esempio i clienti, tassi di cambio, ecc. Una volta in produzione verranno schedulate giornalmente garantendo la sincronizzazione dei dati.

7.4.5 Schema logico

Lo schema logico risultante è visibile in figura **??**.

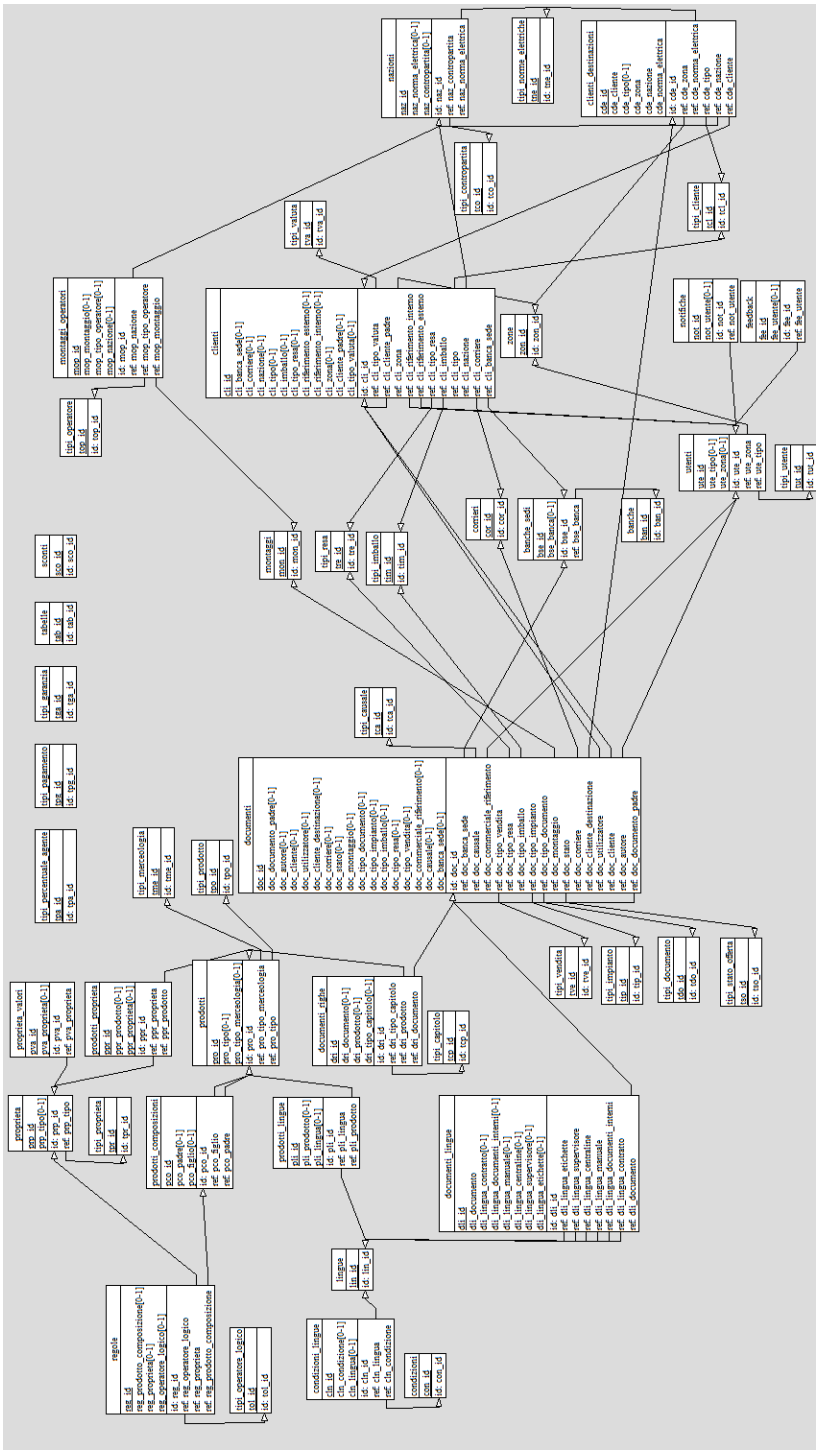


Figura 7.4: Schema logico del database

Capitolo 8

Creazione Model

8.1 Mapping delle classi

Il model dell'applicativo è suddiviso in parte nel package "DatabaseManagement-Package" e in parte, all'interno del package principale. Il package di interfaccia al database è stato realizzato partendo, mediante **EF Core!**, importando le tabelle del database sottoforma di classi entity. Le classi entity sono una rapprosentazione delle tabelle del database, dove ogni classe rappresenta una tabella ed ogni proprietà della classe rappresenta un campo della tabella. Una volta generate le classi entity, si sono individuati gli oggetti fondamentali del dominio, che sono stati mappati in classi **DTO!** mediante il package "Mapster". I **DTO!** sono stati realizzati per essere utilizzati come oggetti neutri che non esponessero direttamente le classi entity, e quindi il database, all'esterno del package. Questi oggetti sono stati scelti in quanto rappresentano le entità principali del dominio, come ad esempio "Cliente", "Prodotto" e "Documento". Le classi **DTO!** sono state realizzate copiando la struttura delle entity di riferimento e, in corrispondenza dei campi legati ad altre tabella mediante foreign key, sono stati adottati due approcci differenti. Se il campo aveva un **DTO!** associato si richiamava quest'ultimo, altrimenti sono stati mappati in oggetti `Dictionary<int,string>`, dove la chiave è l'**ID!** (**ID!**) ed il valore è la descrizione. Il mapping è bidirezionale per poter riconvertire, quando necessario, un **DTO!** in entity per poterlo inserire nel database.

8.2 Struttura del package

Il package è stato strutturato in modo da separare gli ambienti di test da quelli di produzione. All'esterno del package oltre alle classi **DTO!** ed Enum relativi ad alcuni Lookup, è stata costituita una classe univoca che espone i vari repository, denominata "DatabaseManager". La classe "DatabaseManager", si occupa di gestire le connessioni al database, le transazioni e di esporre i vari repository. Questa soluzione permette di avere un unico punto di accesso ai repository, evitando di esporre le classi interne del package e facilitando la **DI!** (**DI!**). La **DI!** è un design pattern che permette di iniettare le dipendenze necessarie ad una classe, senza doverle istanziare all'interno della stessa. Le classi iniettabili vengono registrate all'interno della classe che gestisce la **DI!**, talvolta come singleton, in modo da avere un'unica istanza della classe per tutta la durata dell'applicativo. Il "DatabaseManager" è stato registrato come singleton, mantenendo una sola instanza del database. Ad ogni repository è stata associata una classe di test, che ne testasse in modo efficace i vari metodi. La gestione dei dati dell'utente loggato è stata affidata ad una classe dedicata, anch'essa registrata come singleton, che mantenesse le infomazioni della sessione dell'utente. Tali informazioni includono, ad esempio l'**ID!** dell'utente, il nome e cognome, il ruolo per i permessi. La classe in questione, unita alla tabella *Tabelle*, permette di intercettare la tipologia di richiesta, il livello dell'utente e di conseguenza bloccare operazioni non permesse.

8.2.1 Classi di test

Le classi di test sono state pensate con l'obiettivo di funzionare anche in presenza di dati reali. Per realizzare ciò si sono creati prima di ogni test dei savepoint del database, che permettessero di tornare allo stato iniziale. Successivamente, per ogni test, si sono creati i dati di test fittizi che potessero validare le funzionalità del repository senza il rischio di prelevare dati reali. Questo approccio ha richiesto molto tempo per la ricostruzione dei test più complessi, ma ha permesso di testare piccoli cambiamenti senza dover ripristinare il database. A fronte del tempo impiegato prima si è utilizzata **IA!** generativa per creare le bozze dei test. Tali bozze sono state successivamente modificate ed, in alcuni casi, riscritte completamen-

te, per adattarle al contesto reale. Per realizzare ciò si è adottato un approccio articolato come segue:

1. Si sono scritti i metodi del repository.
2. Si sono generati i test sulla base dei metodi del repository.
3. Si sono corretti i test per renderli accurati e validi.
4. Si sono corretti i metodi del repository sulla base del risultato dei test.

Questo approccio ha permesso di avere un ciclo di sviluppo più veloce, riducendo il tempo di scrittura dei test.

8.2.2 Repository

I repository sono stati divisi in base alle entità principali del dominio, come ad esempio i Prodotti, i Clienti ed i Documenti. Oltre alle operazioni **CRUD!** principali si è scelto, per quanto riguarda le operazioni di ricerca, di utilizzare dei metodi che permettessero di inserire molteplici criteri di ricerca. All'interno sono poi stati utilizzati i `Queryable` per costruire la query in esame. I `Queryable` permettono la costruzione pezzo per pezzo query complesse, includendo solo i criteri necessari e riducendo la complessità delle query. Con un unico metodo si possono così costruire query complesse, evitando di scrivere molteplici metodi per ogni combinazione di criteri. Un esempio di query costruita con `Queryable` è riportata nel listato **??**.

Listing 8.1: Esempio di query Queryable

```
1       public class Example{
2         public async Task<List<UtentiDTO>> GetFilteredUsers(string? username =
              null,
3             string? name = null, string? surname = null, string? email = null, int
                  ? userTypeId = null, bool? deleted = false,
4             CancellationToken cancellationToken = default)
5         {
6             // Check if the user has permission to read UtentiDTO
7             await _permissionService.TryGetTablePermission(typeof(UtentiDTO),
                  TypeOfOperation.READ,
8             _utils.GetCurrentUserRoleId()).ConfigureAwait(false);
9
10 // Build the query to filter users based on the provided parameters
11            IQueryable<Utenti> query = db.Utentis
```

```
                .Where(u => u.Deleted == deleted)
                .Include(u => u.UteTipoNavigation)
                .AsNoTracking();

            if (!string.IsNullOrEmpty(username))
            {
                query = query.Where(u => u.UteUsername!.Contains(username));
            }
            if (!string.IsNullOrEmpty(name))
            {
                query = query.Where(u => u.UteNome!.Contains(name));
            }
            if (!string.IsNullOrEmpty(surname))
            {
                query = query.Where(u => u.UteCognome!.Contains(surname));
            }
            if (!string.IsNullOrEmpty(email))
            {
                query = query.Where(u => u.UteEmail!.Contains(email));
            }

            if (userTypeId.HasValue)
            {
                query = query.Where(u => u.UteTipo == userTypeId.Value);
            }

// Execute the query and retrieve the filtered list of users
            List<Utenti> entities = await query
                .OrderBy(u => u.UteNome)
                .ToListAsync(cancellationToken);

// Map the retrieved list of users to a list of UtentiDTO objects and return it
            return _mapper.Map<List<UtentiDTO>>(entities);
        }
        }
```

Tutti i metodi all'interno dei repository sono stati realizzati come asincroni, per evitare di bloccare il thread principale dell'applicativo e, in particolare, l'interfaccia grafica.

8.3 Gestione dei dati e delle transazioni

Si è deciso di gestire le transazioni in modo atomico, considerandole come commit automatici, per evitare una perdita dei dati in caso di errori o eccezioni. I dati, prima dell'effettivo inserimento nel database, vengono gestiti in memoria **RAM!** e

solo al termine di tutte le operazioni, vengono salvati nel database. Questo approccio permette di non impegnare il database con dati parziali, e di conseguenza, di non dover gestire rollback in caso di errori. Ad ogni metodo dei repository è stato aggiunto il parametro `CancellationToken`[**?**], che permette di forzare l'interruzione dell'esecuzione di una query. L'utilizzo del `CancellationToken` può essere osservato nella query di esempio **??**, dove viene passato come parametro al metodo `FirstOrDefaultAsync()`. La scelta di utilizzare il `CancellationToken` è volta a permettere all'utente di interrompere un'operazione in caso di necessità. L'uso di questo parametro libera le risorse del database e lo rende disponibile per altre operazioni, evitando di bloccare l'applicativo. L'apertura di nuove interfacce richiede quasi sempre una interrogazione al database. L'utilizzo del `CancellationToken` permette di cambiare interfaccia bloccando l'operazione sulla precedente in favore della nuova.

Capitolo 9

Creazione View

9.1 Utilizzo di Figma per disegno interfacce

9.2 Creazione delle Factory per le View

9.3 Scelta delle classi di ViewModel

Capitolo 10

Modulo configurazione prodotto

10.1 Scopo del modulo

10.2 Algoritmo di configurazione

10.3 Realizzazione sul database

10.4 Interfaccia grafica lato utente ed amministratore

Bibliografia

[App] AppMaster. Object transfer data (dto). https://appmaster.io/it/glossary/oggetto-di-trasferimento-dati-dto.

[Fig] Figma. Figma. https://www.figma.com/.

[Map] Mapster. Mapster. https://github.com/MapsterMapper/Mapster.

[Mica] Microsoft. CancellationToken. https://learn.microsoft.com/it-it/dotnet/api/system.threading.cancellationtoken?view=net-10.0.

[Micb] Microsoft. CommunityToolkit.MAUI. https://learn.microsoft.com/it-it/dotnet/communitytoolkit/maui/.

[Micc] Microsoft. MAUI MVVM. https://learn.microsoft.com/it-it/dotnet/architecture/maui/mvvm.

[Micd] Microsoft. Microsoft Entity Framework. https://learn.microsoft.com/en-us/ef/.

[Mice] Microsoft. SQL Server. https://www.microsoft.com/sql-server.

[Micf] Microsoft. SQL Server Express. https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/sql-server-express-localdb?view=sql-server-ver17.

[Micg] Microsoft. SQL Server Integration Services. https://learn.microsoft.com/it-it/sql/integration-services/sql-server-integration-services?view=sql-server-ver17.

[Mich] Microsoft. SQL Server Migration Assistant (SSMA). https://learn.microsoft.com/it-it/sql/ssma/sql-server-migration-assistant?view=sql-server-ver17.

[Mici] Microsoft. XAML language. https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/.

[.NEa] .NET. .NET. https://dotnet.microsoft.com/.

[.NEb] .NET MAUI. .NET MAUI. https://learn.microsoft.com/it-it/dotnet/maui/what-is-maui?view=net-maui-10.0.

[NuG] NuGet. NuGet package manager. https://learn.microsoft.com/it-it/nuget/.

[Rul] Rule Designer. Fusion. https://www.ruledesigner.com/it/.

[Ser] Amazon Web Services. Object relational mapping (orm). https://aws.amazon.com/it/what-is/object-relational-mapping/.

[SQL] SQLite. SQLite. https://www.sqlite.org/. Lightweight database engine.

[Zuc] Zucchetti. Ad-Hoc. https://www.zucchetti.com/it-it/soluzioni/gestionali.

Appendice A

Domande per i gruppi di analisi

In questa appendice sono riportate le domande utilizzate per le interviste ai gruppi di analisi, sia per gli utilizzatori assidui del configuratore commerciale che per i non utilizzatori. Le domande sono state progettate per raccogliere informazioni sulle esperienze degli utenti, le loro aspettative e suggerimenti per migliorare il nuovo configuratore commerciale e i contratti associati.

A.1 Domande per i gruppi di analisi

A.1.1 Domande per entrambi i gruppi

- Anni di esperienza del tuo ruolo?
- Cosa ti aspetti dal nuovo configuratore commerciale e dai nuovi contratti?
- Quali sono le funzionalità attuali più importanti per te?
- Hai suggerimenti per future funzionalità da includere?
- Suggerimenti per facilitare l'adozione del nuovo configuratore?

A.1.2 Domande per utilizzatori assidui

- Quali utilizzi fai dell'attuale programma Access?
- Mostrami un esempio di come utilizzi il configuratore commerciale.

- Quali sono le funzionalità che usi più frequentemente?
- Quali sono le funzionalità che trovi meno utili o difficili da usare?
- Quali sono le principali difficoltà che incontri nell'uso quotidiano?
- Quali operazioni pensi che richiedano più tempo o passaggi rispetto alle procedure che vorresti adottare?
- Puoi descrivere la procedura che hai in mente?

A.1.3 Domande per non utilizzatori

- Come utilizzi attualmente i contratti commerciali nel tuo ruolo?
- Come ti approcci con il reparto vendite?
- Nel tuo ruolo hai bisogno di utilizzare direttamente il configuratore commerciale?
- Raccontami come ti approcci alla gestione dei contratti dopo che arrivano nel tuo reparto.
- Suggerimenti per facilitare l'adozione del nuovo configuratore?

Appendice B

Trascrizione interviste con i soci aziendali

In questa appendice sono riportate le trascrizioni delle interviste effettuate ai soci dell'azienda, al fine di raccogliere informazioni sulle esigenze e le problematiche attuali del sistema di preventivazione.

B.1 Situazione attuale e problematiche

Attualmente l'ufficio commerciale utilizza un database Microsoft Access, in uso da oltre 20 anni.

- **Punti di forza:** Estrema velocità di inserimento per utenti esperti; personalizzazione storica delle stampe e delle logiche.
- **Volume dati:** Circa 6 offerte/giorno per utente (commesse) e 18/giorno (ricambi/**RA!** (**RA!**)). Totale annuo stimato: > 2200 offerte commesse.
- **Limitazioni critiche:**
  - Mancanza di integrazione con il gestionale **ERP!** (Ad Hoc).
  - Necessità di reinserimento dati manuale da Offerta a Ordine (doppio lavoro).

  - Gestione prezzi manuale o basata sull'esperienza (non collegata al costo industriale aggiornato).

B.2 Criticità rilevate

B.2.1 Disallineamento tecnico-commerciale

Attualmente, una volta firmato il contratto commerciale, le modifiche tecniche (**PM!** (**PM!**)/**UT!** (**UT!**)) non vengono tracciate nel sistema di preventivazione.

- **Conseguenza:** Per replicare un impianto o vendere ricambi corretti, il commerciale deve consultare gli schemi CAD/Tecnici e il gestionale, poiché l'offerta originale in Access diventa obsoleta/inaccurata dopo la conferma d'ordine.
- **Rischio:** Vendita di ricambi errati o incoerenze nelle offerte copiando vecchie offerte per recuperare informazioni, condizioni e note.

B.2.2 Inefficienza Gestione Ricambi (Spare Parts)

Il flusso per i ricambi è eccessivamente complesso rispetto al valore economico.

- **Situazione attuale:** L'Area Manager deve aprire una nuova offerta da zero o copiare vecchie offerte, richiedere quotazioni interne se non già presenti, attendere risposte e infine emettere l'offerta. Dopo la conferma del cliente, deve trasformarla in ordine.
- **Collo di bottiglia:** Troppi passaggi (Commerciali → Ufficio Tecnico → Pricing Office → Offerta).
- **Obiettivo:** Ridurre drasticamente i tempi per rendere profittevole la vendita proattiva, che ad oggi non viene effettuata.

B.3 Requisiti funzionali emersi

B.3.1 Gestione offerte e configuratore

Il sistema deve guidare l'utente, specialmente se junior, nella composizione dell'impianto.

- **Selezione guidata:** Selezionando un macro-componente (es. "Trasporto Pneumatico" o "STX"), il sistema deve proporre in automatico gli accessori correlati (filtri, valvole, compressori) tramite menu a tendina.
- **Alert di completezza:** Avvisi automatici in caso di macchine essenziali mancanti (es. "Hai inserito il trasporto, ma manca il compressore").
- **Template per applicazione:** Possibilità di richiamare configurazioni standard (Template di impianto) basate sul prodotto finale del cliente (es. "Linea Pan di Spagna", "Linea Biscotti") e non solo sul nome cliente.

B.3.2 Listini e pricing

- **Automazione costi:** Il prezzo di vendita deve scaturire automaticamente dal costo industriale aggiornato (integrazione con il gestionale), applicando regole di ricarico predefinite.
- **Eliminazione colli di bottiglia:** Il commerciale non deve dover chiedere quotazioni interne per componentistica standard.

B.3.3 Gestione flusso obbligatorio ordine e revisioni

- **Integrazione ERP:** Passaggio automatico da "Offerta Confermata" a "Ordine Gestionale" senza reinserimento dati.
- **Doppio contratto (Versionamento):**
  1. **Contratto commerciale:** La versione firmata dal cliente (congelata).
  2. **Contratto tecnico ("As-Built"):** Una copia viva che viene modificata dai PM/Ufficio Tecnico durante la realizzazione. Questa diventerà la base per le future assistenze e i ricambi.

B.3.4 Distinzione flussi di lavoro

Il software deve distinguere a monte la tipologia di richiesta per instradare il lavoro:

1. **Nuovi impianti / commesse:** Flusso standard completo.
2. **Ampliamenti:** Collegati a commesse esistenti ma gestiti dai commerciali.
3. **Ricambi (spare parts):** Flusso semplificato e rapido. Accesso allo storico installato.

B.4 Requisiti non funzionali

- **Modalità offline:** Fondamentale per gli Area Manager in viaggio in zone con scarsa connettività. Sincronizzazione al rientro o via **VPN!**.
- **Multilingua:** Gestione avanzata delle descrizioni (necessità di una via di mezzo tra la descrizione "estesa" attuale, troppo lunga, e quella "breve", troppo criptica).
- **Velocità:** L'interfaccia non deve rallentare l'operatività dell'utente.

Appendice C

Trascrizione utilizzatori

In questa appendice sono riportate le trascrizioni delle interviste condotte con gli utilizzatori abituali del sistema attuale, al fine di raccogliere i requisiti per il nuovo software.

C.1 Operazioni comuni

Le operazioni più comuni svolte all'interno dell'applicativo includono:

- Creazione di offerte per ricambi e assistenza (**RA!**).
- Creazione di offerte per clienti non diretti.
- Creazione delle offerte Access per conto di agenti o rivenditori esterni.
- Creazione di offerte per clienti diretti.
- Creazione degli ordini ogni qualvolta un'offerta viene accettata.
- Ricerca e copia di vecchie offerte simili durante la creazione di nuove offerte.
- Correzione delle traduzioni preimpostate fornite da Access.

C.2 Gestione protezione dati e accessi

C.2.1 Gestione utenti e visibilità degli agenti

Per quanto riguarda la gestione degli utenti e la visibilità dei dati, gli agenti hanno evidenziato i seguenti punti chiave:

- **Segregazione dei dati:** È fondamentale mantenere la distinzione tra offerte e **RA!** (Richieste assistenza/ricambi). La distinzione tra i due tipi aiuta a evitare confusione e garantisce che le operazioni siano gestite correttamente.
- **Privacy per autore:** Un agente esterno non deve vedere i progetti o le offerte degli altri agenti per evitare conflitti di interesse (Policy aziendale).
- **Anagrafica clienti condivisa:** A differenza delle offerte, l'elenco clienti deve essere visibile a tutti, può capitare di fare offerte per conto di altri commerciali.

C.3 Flussi di lavoro attuali

C.3.1 Ricambi e assistenza post-vendita

Il flusso di lavoro attuale, per la gestione dei ricambi e dell'assistenza post-vendita, presenta diverse inefficienze dovute alla frammentazione degli strumenti e alla mancanza di dati strutturati.

- **Template di partenza:** Attualmente non esiste un template dedicato; si parte quindi da vecchi ordini e, per individuare quali siano le assistenze, l'unica discriminante è l'autore. Questa pratica permette di ereditare tutte le condizioni di vendita e i dati del cliente, risparmiando tempo nella compilazione (operazione attualmente molto onerosa in termini di tempo).
- **Ricerca ricambi manuale:** L'operatore deve consultare disegni tecnici (Fusion, PDF, cartacei) e database separati (Access, Ad-Hoc, Excel, Server "Commesse") per identificare i codici dei ricambi.

- **Inserimento del componente:** Una volta individuato il componente, non essendoci una corrispondenza univoca dei codici, l'operatore deve trovare la corrispondenza in Access o creare un nuovo codice per il ricambio.
- **Creazione offerta complessa:** Non è facile né la ricerca né l'assegnazione di un prezzo se il componente è datato; la compilazione dell'offerta talvolta richiede calcoli manuali (prezzi, pesi, volumi). Spesso si ricorre alla copia di vecchie offerte **RA!** o commesse, pratica che trascina errori pregressi.

C.3.2 Impianti nuovi e ampliamenti

L'attuale processo di creazione delle offerte prevede i seguenti passaggi:

1. **Raccolta requisiti:** Il commerciale si interfaccia con il cliente per comprendere le esigenze tecniche e commerciali.
2. **Ricerca e copiatura di una vecchia offerta:** Spesso si parte da un'offerta precedente simile per velocizzare il processo.
3. **Selezione prodotti:** Aggiunta manuale dei prodotti, uno a uno, tramite descrizione.
4. **Configurazione offerta:** Creazione dell'offerta suddivisa in capitoli, applicando gli sconti e i calcoli necessari.
5. **Invio al cliente:** Generazione del documento finale in formato PDF o cartaceo e invio al cliente.
6. **Approvazione offerta:** In caso di accettazione, si procede con la conferma d'ordine e la comunicazione all'ufficio produzione e all'ufficio amministrativo.

C.4 Problemi attuali del sistema

Dall'intervista i commerciali hanno fatto emergere le seguenti criticità del sistema attuale:

- **Lentezza e complessità:** L'intero processo di gestione è lento e laborioso e richiede molto tempo per completare un'offerta. Questo è dovuto alla necessità di consultare molteplici fonti di dati non integrate. Il programma stesso utilizzato (Access) risulta lento e macchinoso, con molti tempi morti.
- **Disallineamento codici:** Esiste una discrepanza tra i codici commerciali e quelli tecnici/gestionali. Questo costringe a ricerche manuali su schemi funzionali o database separati per identificare il pezzo corretto (Ad-Hoc, Fusion, ...).
- **Mancanza di automazione nei calcoli:**
  - **Prezzi:** I costi e i prezzi di vendita devono essere spesso calcolati manualmente o verificati su Excel/Ad-Hoc, specialmente per componenti su misura o obsoleti.
  - **Logistica:** Non esiste un calcolo automatico di volumi e pesi per il trasporto; l'utente deve ipotizzare i dati basandosi su vecchi DDT o sull'esperienza. Il costo è talvolta calcolato mediante il sito del corriere.
  - **Manodopera:** I tempi e i costi per gli interventi tecnici (es. sostituzione saccone) non sono standardizzati e richiedono consultazioni continue con i colleghi (es. Logistica).
- **Componenti elettrici:** La gestione e l'inserimento di materiale elettrico sono molto difficili per la complessità e la varietà di codici esistenti. Attualmente si consulta direttamente l'ufficio tecnico elettrico per avere un elenco dei componenti necessari da offrire.
- **Gestione obsolescenza:** È difficile sapere se un componente venduto anni fa è stato sostituito da una nuova versione o se è obsoleto, rischiando di vendere pezzi non più procurabili. Serve un sistema di tracciamento delle sostituzioni e delle obsolescenze (es. Fusion).
- **Discrepanza "As Built":** Le modifiche fatte in cantiere (**RA!** di cantiere) o successive alla vendita non sono sempre tracciate in modo unitario, rendendo difficile conoscere la configurazione reale dell'impianto attuale, punto fondamentale per una corretta assistenza senza errori.

- **Sistema di traduzione:** È fondamentale avere un sistema di traduzione integrato e affidabile per le descrizioni dei componenti e delle offerte; attualmente, oltre alle traduzioni preimpostate di Access (spesso da correggere), si utilizzano servizi esterni (Google Translate) che richiedono tempo e possono introdurre errori.

C.5 Requisiti Funzionali per il nuovo software

C.5.1 Database e integrazione

Per migliorare l'integrazione dei dati e la gestione delle informazioni, il nuovo sistema deve prevedere:

- **Database unificato:** Il nuovo sistema deve integrare o dialogare con Ad-Hoc **ERP!**, Fusion **PLM!** e l'archivio storico per garantire l'univocità del dato.
- **Codifica condivisa:** Allineamento tra codici commerciali e tecnici. Il codice usato per l'offerta deve permettere di risalire automaticamente a costi, distinta base e disegni.
- **Gestione revisioni:** Il sistema deve tracciare le revisioni dei clienti (cambi nome, stabilimento) e dei componenti (sostituzioni, obsolescenza).
- **Codici a catalogo:** Il sistema deve già contenere tutti i codici esistenti; la volontà del commerciale è di non doverli inserire manualmente. È necessario prevedere una gestione esterna dedicata all'aggiunta e all'aggiornamento di nuovi codici.
- **Integrazione nel flusso dell'ordine:** Attualmente esistono 3 codici fondamentali per la gestione dell'ordine:
  - **Codice offerta:** Codice univoco dell'offerta commerciale.
  - **Codice ordine:** Codice univoco dell'offerta trasformata in ordine.
  - **Codice distinta:** Codice univoco della distinta base gestionale dell'ordine.

Il nuovo sistema deve collegare e mostrare questi 3 codici in modo chiaro e visibile per l'operatore, facilitando le ricerche e la tracciabilità dell'ordine.

C.5.2 Ricerca e navigazione ricambi

Per facilitare la ricerca e la selezione dei ricambi, il sistema deve prevedere:

- **Ricerca multicriterio:** Necessità di cercare per Numero di Commessa (fondamentale), Cliente, Stabilimento e Tag Tecnici (es. "valvola a farfalla").
- **Struttura ad albero/gerarchica:** Possibilità di esplorare la distinta base **BOM!** (**BOM!**) scendendo nei livelli (esplosione componenti) per individuare il singolo ricambio.
- **Filtri per "Ricambi tipici":** Implementazione di filtri che mostrino, per una data famiglia di macchine (es. Lift, Vibrovaglio), solo i componenti tipicamente venduti come ricambi; i codici devono essere allineati ai codici tecnici, velocizzando l'assistenza una volta individuato il componente.

C.5.3 Creazione e gestione offerte

Per semplificare e velocizzare la creazione delle offerte di ricambi e assistenza, il sistema deve prevedere:

- **Automazione descrizioni e lingue:** Compilazione automatica delle descrizioni in lingua (traduzioni pre-caricate) senza dover usare traduttori esterni.
- **Calcolo prezzi automatizzato:** Il sistema deve recuperare il costo industriale da Ad-Hoc e applicare le maggiorazioni o i listini specifici per cliente/settore.
- **Tracciabilità univoca della pratica:** Mantenimento di un riferimento univoco che segua la pratica dalla richiesta iniziale all'ordine confermato, per evitare duplicazioni e facilitare la rintracciabilità, affiancato da uno stato di avanzamento *(es. "In Revisione", "Inviata", "Accettata", "Rifiutata")* per facilitarne l'interpretazione.

- **Distinzione tipologie RA:** Gestione differenziata ma tracciabile per "Ricambi", "Ampliamenti" e "RA di Cantiere". Al momento le assistenze sopra citate sono raggruppate sotto l'unico cappello del "VSS".

C.5.4 Funzionalità avanzate e UX

Per migliorare l'esperienza utente e l'efficienza operativa, il sistema deve prevedere:

- **Intestazione offerta e dati cliente:** È importante riportare tutte le informazioni attuali del cliente (Indirizzo, Nome, Numero di commessa associata) in modo chiaro e visibile nella schermata dell'offerta.
- **Schermata dedicata allo storico:** Una sezione dove poter visualizzare tutte le offerte e gli ordini precedenti associati al cliente o alla commessa selezionata.
- **Schermata offerta intuitiva:** Interfaccia user-friendly che guidi l'utente passo-passo nella compilazione dell'offerta, minimizzando errori e omissioni.
- **Visualizzazione dati completa:** Interfaccia che mostri tutte le informazioni rilevanti (es. dettagli tecnici completi come "filtrante/non filtrante") senza dover scorrere o aprire popup.
- **Modalità offline/sync:** Il software deve permettere di lavorare in locale (offline) durante le visite dai clienti e sincronizzarsi automaticamente con il database centrale una volta ristabilita la connessione VPN.
- **Multi-finestra:** Possibilità di tenere aperte più schede o finestre di offerte diverse contemporaneamente. Utile la possibilità di un sistema multi-documento *Es: AutoCAD.*
- **Interfaccia mobile/tablet:** Ottimizzazione per l'uso su tablet (iPad) per la compilazione di dati preliminari e note durante il sopralluogo o in fiera.

C.6 Note di implementazione

Gli utilizzatori hanno fornito alcune indicazioni utili per la fase di implementazione:

- **Campi obbligatori:** Rendere obbligatoria la distinzione tra "Ampliamento" e "Ricambi" (attualmente campo discrezionale spesso vuoto) per migliorare la ricercabilità futura.
- **Beta tester:** Si consiglia di iniziare la fase di test con figure tecniche/analitiche e commerciali esperti.
- **Transizione:** Prevista una fase transitoria con utilizzo parallelo dei due sistemi; necessaria migrazione o accessibilità facile ai dati storici.

C.7 Numeri utili

Per capire gli attuali tempi di lavoro e il volume storico di offerte gestite, si riportano i seguenti dati:

- **Offerte VS Ordini:** Attualmente a sistema risultano create (o presenti nello storico **RA!**) circa 57.000 offerte (**RA!**), di cui circa 41.000 si sono trasformate in ordini effettivi.
- **Tempo base per una semplice offerta:** Attualmente, per una richiesta tipo di cui si conoscono già il numero di commessa e i codici da inserire, il tempo medio per la creazione di un'offerta è di circa 15 minuti. Il tempo sale notevolmente se devono essere ricercate le informazioni mancanti.

Appendice D

Trascrizione non utilizzatori

In questa appendice sono riportate le trascrizioni delle interviste effettuate ai non utilizzatori del sistema di vendite attuale. Le interviste hanno avuto l'obiettivo di raccogliere informazioni sulle esigenze odierne e future del sistema di preventivazione..

D.1 Ruoli coinvolti e utilizzo attuale degli strumenti

I reparti tecnici, di gestione commessa e logistici interagiscono in modo indiretto o marginale con l'attuale database Microsoft Access, basandosi prevalentemente su passaggi cartacei o file esterni:

- **Ufficio Project Management e Project Engineering:** Coordinano le commesse tra commerciale e reparti tecnici, definendo le specifiche e supervisionando installazione e post-vendita. Non utilizzano Access; consultano i contratti in formato cartaceo o scansionato per revisionare le specifiche tecniche e contattare i clienti.
- **Ufficio Tecnico Meccanico:** Si occupa della progettazione meccanica e dei disegni costruttivi. Non utilizza Access; recepisce le specifiche dal cartaceo tramite i **PE!** (**PE!**) per verificare revisioni, limiti dei componenti e condizioni del sito di montaggio.

- **Ufficio Tecnico Elettrico:** Gestisce la progettazione di quadri e impianti elettrici. Attualmente sopperisce alle carenze del sistema utilizzando fogli Excel esterni per il dimensionamento/calcolo costi (*QUAEL.xltm*) e per la validazione tecnica dei preventivi (*PROGETTO CEPI - Scheda progetto r2.xlsx*).

- **Ufficio Logistica:** Gestisce spedizioni, dogane e trasporti. Utilizza Access solo marginalmente per verificare le date di consegna o correggere a valle indirizzi, causali, dati di trasporto e Incoterms. L'operatività ruota attorno a un "File Logistico" in Excel centralizzato e alla gestione di copie cartacee nelle cartelle di rete.

- **Ufficio Manualistica e Ufficio Acquisti:** La Manualistica redige manuali e certificazioni dei prodotti venduti; l'Ufficio Acquisti gestisce gli ordini ai fornitori e controlla i contratti commerciali per verificare i ricambi in assistenza ed eventuali discrepanze tecniche o commerciali.

D.2 Criticità trasversali rilevate

D.2.1 Frammentazione documentale e supporto cartaceo

- **Disallineamento delle informazioni:** L'utilizzo di copie cartacee multiple (distribuite tra Vendite, Ufficio Tecnico, Elettrico e Logistica) e di annotazioni manuali sul cartaceo comporta la perdita di informazioni e la circolazione di versioni obsolete.

- **Duplicazione per tempistiche differenti:** Per le commesse miste (meccaniche ed elettriche), la Logistica è costretta a duplicare fisicamente le copie tecniche delle **RA!** per gestire le diverse tempistiche dei due uffici.

D.2.2 Disconnessione anagrafica ed errori di input

- **Disallineamento Access vs Ad Hoc:** Access presenta anagrafiche spesso non aggiornate e prive di storicizzazione, mentre l'ERP Ad Hoc contiene i

dati fiscali corretti (Sede Legale) ma risulta impreciso sulle sedi operative o di spedizione.

- **Propagazione degli errori:** La prassi commerciale di duplicare vecchie commesse/RA trascina errori pregressi che devono essere corretti manualmente dalla Logistica a valle del processo.
- **Omissione dati logistici:** Nelle offerte mancano quasi sempre informazioni essenziali per il trasporto quali Incoterms, tipologia di imballo, pesi e volumi.

D.2.3 Gestione post-vendita e disallineamento per le dogane

- **Variazioni non tracciate:** Il metodo attuale punta alla vendita rapida, ma non traccia in modo strutturato i componenti aggiunti o modificati in fase di progettazione e realizzazione.
- **Criticità sulle Packing List doganali:** I commerciali vendono "macro-codici" (es. impianto completo), ma le dogane (specialmente extra-UE) esigono il dettaglio di ogni singolo pallet. Poiché il sistema non esplode i macro-codici nei colli fisici, l'associazione tra voci contrattuali, distinta tecnica e pallet preparati dagli spedizionieri avviene manualmente, generando un alto rischio di errori doganali.

D.3 Requisiti funzionali emersi

D.3.1 Struttura del contratto, distinta base e codifica

- **Gestione per macro-codici e standardizzazione:**
  - Utilizzo di "macro-codici" contenitore in fase di vendita (es. *Macro-codice Quadro*, *Pannello Operatore*, *TVB generico*) direttamente agganciati alla distinta base (completa al 90%), che l'Ufficio Tecnico completerà poi con i disegni specifici.

  - Spinta verso la "standardizzazione a catalogo" tramite l'analisi statistica dei *Best Sellers* (es. TVB standard, stellari taglie 1000/1500/2000), facendo coincidere il codice di vendita con quello di realizzazione e separando il flusso standard da quello "fuori standard".
  - Allineamento univoco dei codici articolo tra il nuovo configuratore e il gestionale Ad Hoc.

- **Separazione ed esplosione delle voci (Meccanica vs Elettrica):**
  - Creazione di due sezioni contrattuali ben distinte per i componenti meccanici e quelli elettrici, superando la confusione attuale.
  - Esplosione automatica: l'inserimento di utenze meccaniche (es. motori, vibratori) deve generare automaticamente le voci elettriche correlate nel contratto tecnico (es. gli inverter devono essere integrati nel quadro o associati all'utenza specifica, non trattati come voci isolate).

- **Rintracciabilità e collegamento logistico:**
  - Superamento dei codici generici per garantire la tracciabilità dell'esatta variante del componente venduto (es. variante specifica di un *WP104*).
  - Collegamento nativo tra distinta commerciale e distinta tecnica (tramite Rif CC) fin dalla progettazione, per automatizzare l'associazione con i pallet fisici e la generazione delle Packing List.

D.3.2 Configuratore tecnico, normativo e logistico

Il sistema deve guidare il commerciale attraverso vincoli all'ingresso (menu a tendina) e controlli automatici di completezza prima del salvataggio:

- **Guida e vincoli per la parte elettrica:**
  - Suggerimento automatico del materiale del quadro in base alle utenze inserite (es. taglia corretta dell'inverter in base ai motori venduti) e domande finali per proporre accessori consigliati.

  - Inserimento automatico di default dei sezionatori a bordo macchina per ogni motore.
  - Distinzione netta tra “quadro nuovo” e “ampliamento”: in caso di ampliamento, il sistema deve indicare la disponibilità di spazio fisico nel quadro esistente per evitare errori di valutazione e costi imprevisti.
  - Campi tecnici obbligatori: Paese di destinazione, alimentazione (es. 230V 50Hz, 400V 60Hz, DC/AC), materiale del quadro (inox, verniciato), tipologia canaline (chiusa, forata), numero/tipologia utenze e certificazioni (es. UL/USA, la cui selezione deve aggiornare in automatico componenti e costi).

- **Specifiche per manualistica, meccanica e acquisti:**
  - Campi dedicati alle certificazioni richieste per i componenti venduti (CE, ATEX, FDA, ecc.) e alle norme di riferimento per i prodotti acquistati da terzi (ATEX, alimentari, ecc.).
  - Valorizzazione dettagliata delle caratteristiche tecniche (dati elettrici dei motori, dati meccanici, allestimenti, accessori) per agevolare la stesura dei manuali.
  - Verifica delle condizioni del luogo di montaggio per attivare controlli specifici sulle macchine (es. Globosilo esterni in zone sismiche), definire i trasporti e rilevare limitazioni d’uso dei componenti.

- **Vincoli e alert logistici/doganali per Paese:**
  - Compilazione obbligatoria e guidata di Incoterms (DAP, DDP, FCA, ecc.), tipologia di imballo, pesi e volumi.
  - Generazione di alert automatici all’inserimento dell’ordine in base al Paese di destinazione per requisiti di fumigazione/ISPM 15 sui pallet in legno (es. Brasile, Australia, Turchia), materiali proibiti (es. ancoranti chimici vietati) e necessità di permessi eccezionali o montaggi.

D.3.3 Preventivazione, calcolo costi e varianti

- **Internalizzazione dei fogli di calcolo:** Integrazione nel software delle logiche di dimensionamento, calcolo costi e validazione preventiva oggi gestite sui file Excel esterni (*QUAEL.xltm* e *Scheda progetto r2.xlsx*).
- **Costificazione delle variazioni:** Possibilità per il Project Management di costificare le variazioni tecniche in modo separato dal contratto commerciale, così da valutare il margine di manovra sulle modifiche in corso d'opera.

D.3.4 Gestione contrattuale, revisioni e "As-Built"

- **Doppio contratto (Commerciale vs Tecnico):** Poiché il contratto commerciale firmato e sdoganato ha vincoli legali e non può essere alterato liberamente, il sistema deve gestire in parallelo un "Contratto Tecnico" (copia speculare).
- **Allineamento Contratto vs Schema (P&ID) e Feedback Loop:**
  - Sostituzione di note esterne e file Word con una sezione dedicata che registri le discrepanze tra venduto commerciale ed eseguito tecnico (es. modifiche allo schema P&ID, numero valvole, componenti extra), mantenendo allineati i dati tecnici ed economici.
  - Implementazione di un sistema di feedback *"As-Built"* per registrare le modifiche effettive apportate durante la realizzazione, ottimizzando i processi e la precisione dei futuri preventivi.
  - Possibilità di riproporre al cliente il contratto tecnico aggiornato affinché diventi il documento ufficiale.

D.3.5 Workflow, pianificazione temporale e notifiche

- **Monitoraggio date e colli di bottiglia:** Registrazione e confronto automatico tra la *Data emissione ordine* (conferma cliente) e la *Data stampa/presa in carico* (ricezione interna), evidenziando eventuali ritardi.

- **Pianificazione a ritroso (Back-Scheduling):** Calcolo automatico delle milestone intermedie a partire dalla *Data di evasione* (spedizione prevista) o dallo *Start-up impianto* per garantire il rispetto delle tempistiche.
- **Gestione stati e Alerting automatico:** Distinzione chiara degli stati della commessa (es. da *Pre-inserimento / Bozza commerciale* a *Inserito in Impianti / Validato tecnicamente*). Al cambio di stato, invio di notifiche automatiche (email o alert in-app) ai reparti a valle (es. Elettrico, Logistica), sostituendo il passaggio fisico del fascicolo cartaceo e le email manuali.

D.4 Requisiti su anagrafiche, tracciabilità e governance

- **Tripla entità anagrafica e storicizzazione:**
  - Distinzione obbligatoria in anagrafica tra tre soggetti: *Intestatario* (Fiscale), *Utilizzatore* (Fisico) e *Destinatario Merce* (Logistico).
  - Storicizzazione dei dati anagrafici e introduzione di *date di obsolescenza* per articoli e sedi: in caso di cambio sede o P.IVA del cliente, i documenti storici (es. DDT passati) devono conservare i dati validi all'epoca dell'emissione.
- **Tracciabilità totale (Audit Log) e responsabilità:**
  - Eliminazione delle annotazioni su carta a favore di una visibilità digitale in tempo reale per tutti i reparti autorizzati (Commerciale, **UT!**, **PM!**, Logistica, Magazzino).
  - Log completo delle modifiche ai dati di commessa che registri *Chi* ha effettuato la variazione, *Quando* (timestamp) e *Cosa* è cambiato (valore precedente vs valore attuale).
  - Presenza di un campo "Compilato da" autocompilato con l'utente loggato, ma mantenuto editabile per gestire eventuali deleghe o correzioni di assegnazione.

- **Governance Top-Down e formazione:**
  - Impostazione rigida del database: divieto di popolamento libero da parte degli utenti e definizione delle regole a monte tramite un team ristretto per preservare l'integrità dei dati.
  - Formazione specifica per il reparto commerciale sull'utilizzo del nuovo sistema e sull'importanza della correttezza dei dati inseriti a monte.

Acknowledgements

Optional. Max 1 page.